

# SIH INTERNAL HACKATHON: LEVEL 3 Problem Statements (High Priority)

All project names, ObjectIds, schema names, function names, API URLs, and tokens in this document are fictional sample data. No real credentials, production identifiers, or proprietary code may appear in student submissions.

| Topic                                   | Topic Title                                                | Mandatory Tech Stack                        |
|-----------------------------------------|------------------------------------------------------------|---------------------------------------------|
| Topic 1<br>(Problem Statement ID: PS7)  | AI-Assisted UI Generation from Wireframe, Code, and Prompt | React.js   Node.js                          |
| Topic 2<br>(Problem Statement ID: PS8)  | Binding Automation                                         | MongoDB + Node.js                           |
| Topic 3 (Problem Statement ID: PS9)     | Dynamic Function Creation                                  | Node.js   MongoDB   React.js                |
| Topic 4<br>(Problem Statement ID: PS10) | Business-Context API Test Generation                       | Node.js (TypeScript)   Playwright   MongoDB |
| Topic 5<br>(Problem Statement ID: PS11) | Business Workflow Detection & Diagram Generation           | Node.js   MongoDB   React.js                |
| Topic 6<br>(Problem Statement ID: PS12) | Universal AI Chat Interface                                | Node.js   MongoDB   React.js                |

# Problem Statement ID – PS7: AI-Assisted UI Generation from Wireframe, Code, and Prompt

**Note:** All brand names, IDs, file paths, storage URLs, commit hashes, and production identifiers in this document are fictional sample data. Do not use real client, environment, or credential values.

## 1. Document Control

| Field           | Value                                                     |
|-----------------|-----------------------------------------------------------|
| Document title  | UI Generation (React.js + Node.js)                        |
| Classification  | Internal hackathon use - sample data only                 |
| Tech constraint | Frontend: React.js. Backend: Node.js. Both are mandatory. |

### 1.1 Purpose of this document

This document defines the problem, scope, data contracts, functional requirements, evaluation rubric, and sample artefacts for a student hackathon. Teams will build a system that generates CMS-ready React UI sections from one or more of three inputs: a wireframe image, existing source code, and a natural-language prompt.

## 2. Executive Summary

Modern websites are no longer static pages. Marketing and product teams need sections that can be edited in a CMS after generation: headlines, images, stats, and buttons must have stable element IDs, default fallback content, and a Redux (or equivalent) data binding layer.

The hackathon challenge is to build a generator — not a one-off landing page. Given a wireframe, a code snippet, a prompt, or a combination of these, the system must emit:

- 1. A React section component that follows the sample architecture in Appendix C.
- 2. A section metadata record (Appendix A).
- 3. A list of CMS element records (Appendix B).
- 4. A live preview of the generated section.

Success is measured by layout fidelity, CMS contract compliance, code quality, and the ability to regenerate or vary a section without hand-rewriting IDs and bindings.

## 3. Problem Statement

Converting a design into a production-ready, content-managed React section is slow and inconsistent. Typical issues:

- Wireframes describe layout, but developers invent different class names, spacing, and component structure.

- Prompts describe intent, but generated JSX is often presentational only — hard-coded text, no IDs, no CMS fetch.
- Existing section code is difficult to reuse because IDs, Redux selectors, and default content are tightly coupled.
- Repeatable blocks (stat cards, feature lists) need a loop schema, not three unrelated hard-coded blocks.

Students must close this gap: an automated pipeline that understands visual structure, textual intent, and code patterns, then outputs a section that a CMS editor could later change without touching React source.

## 4. Hackathon Objectives

### 4.1 Primary objectives (must achieve)

- Generate a visually faithful React section from at least two of the three input types (wireframe, code, prompt).
- Bind every editable field to a unique fieldId and render CMS content with a hard-coded default fallback.
- Persist section metadata and element documents in a Node.js API with a document store (MongoDB, or a JSON file store if MongoDB is unavailable).
- Preview the generated section in the browser using Redux (or equivalent) as the runtime content source.

### 4.2 Secondary objectives (strongly recommended)

- Accept combined inputs (example: wireframe + prompt that says “make CTA green and use 3 stat cards”).
- Support a second variation of the same section (variations field in section metadata).
- Re-generate from existing code while preserving element IDs when the layout change is small.

### 4.3 Stretch objectives (bonus)

- Vision model that labels regions of the wireframe (hero image, badge, H1, stats, CTA).
- CSS overlay per fieldId (allSectionsCss) applied after mount.
- Export a zip of the generated component plus seed JSON for section and elements.
- Simple content editor in the preview that writes back to the element store.

## 5. Scope

### 5.1 In scope

- A student-built generator UI (React) and generator API (Node.js).
- One reference page: Home. One reference section: a split hero (image left, content right).
- Element types: Image, Text, Textfield, Button, Cards (with loop items), carousel, wizards, etc.
- Responsive layout using Tailwind CSS utility classes.
- PrimeReact Button (or equivalent accessible button) for the primary CTA.
- Placeholder images and fictional brand copy only.

### 5.2 Out of scope

- Real client brands, real production databases, real storage credentials, and real commit / deploy pipelines.
- Full multi-tenant CMS, authentication SSO, payments, or analytics.
- Native mobile apps (platform for this topic is Website).
- Pixel-perfect Figma plugin work unless a team chooses it as a stretch extra.
- Training a foundation model from scratch.

## 6. Target Users and Personas

| Persona             | Goal during the hackathon                   | Success looks like                                        |
|---------------------|---------------------------------------------|-----------------------------------------------------------|
| Student builder     | Ship a working generator + preview          | Judges can upload a wireframe and see a CMS-bound section |
| Mock content editor | Change headline / image without editing JSX | Updating an element record updates the preview            |
| Mentor / judge      | Score contract compliance, not only beauty  | IDs, fallbacks, cards loop, and APIs are demonstrable     |

## 7. Mandatory Technology Stack

| Layer              | Required                                          | Allowed alternatives / notes                                                                        |
|--------------------|---------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| Frontend framework | React.js (functional components)                  | Vite recommended. Create React App is acceptable.                                                   |
| UI styling         | Tailwind CSS for layout                           | May combine with a small amount of CSS modules.                                                     |
| Component library  | PrimeReact for Button (and optional form widgets) | If PrimeReact fails to install, use a semantic <code>&lt;button&gt;</code> with equivalent classes. |
| Client state       | Redux Toolkit for CMS section/element maps        | Zustand allowed only if the same selector shape is documented.                                      |
| Routing            | react-router-dom                                  | At least <code>/preview/:pageName</code> and <code>/generate</code> .                               |
| Backend runtime    | Node.js                                           | Express, Fastify, or NestJS are all acceptable.                                                     |
| Persistence        | MongoDB or file-based JSON store                  | Must expose REST (or GraphQL) for sections and elements.                                            |
| File upload        | Multipart upload for wireframe images             | Store files locally under <code>/uploads</code> during the event.                                   |
| AI / LLM           | Any hosted or local model                         | Must not leak API keys in the repo. Use <code>.env</code> (not committed).                          |
| Language           | JavaScript or TypeScript                          | TypeScript is preferred but not mandatory.                                                          |

### 7.1 Environment variables (sample names only)

```
VITE_STORAGE_URL=http://localhost:4000/storage/  
VITE_API_URL=http://localhost:4000/api  
MONGODB_URI=mongodb://localhost:27017/hackathon_ui_gen
```

```
LLM_API_KEY=replace-me
LLM_BASE_URL=https://example-llm-provider.local/v1
```

## 8. Challenge Definition

Teams build a UI Generation Studio. A user supplies one or more inputs. The studio returns a React section that matches the visual structure and is wired for CMS editing.

### 8.1 Input mode A — Wireframe

- The user uploads a PNG/JPG/WebP of a section wireframe (low-fidelity boxes or a screenshot-style mock). The system infers regions: media, eyebrow/badge, primary heading, subheading, body copy, repeating stats/cards, and primary action.
- Accept at least PNG and JPG.
- Show the uploaded wireframe beside the generated preview for visual comparison.
- If a region is uncertain, still emit a plausible element with a default and mark confidence in the API response (optional but scored).

### 8.2 Input mode B — Existing code

- The user pastes a React section (or a simplified JSX tree). The system must parse structure, extract text/image/button nodes, assign or reuse fieldIds, and emit a CMS-compliant component plus element JSON. This mode tests understanding of the contract, not only image-to-layout.
- Detect hard-coded strings and lift them into element records with defaults.
- Detect repeating children and convert them into a Cards loop.
- Preserve Tailwind layout classes when they already express the wireframe.

### 8.3 Input mode C — Prompt

- The user types intent, for example: “Create a fitness hero. Left image of an athlete. Right side: red uppercase badge, bold headline, supporting paragraph, three stats, full-width red CTA.” The system must produce the same contract as modes A and B.
- Prompt-only generation is valid, but layout should still be responsive (stacked on mobile, split on desktop).
- The prompt may override colours, copy, number of cards, and CTA label.

### 8.4 Combined inputs (recommended)

- 1. Prompt instructions win for copy, colour, and CTA behaviour.
- 2. Wireframe wins for spatial layout (columns, order, alignment).
- 3. Existing code wins for technical patterns (Redux selectors, helper names, class conventions) unless the prompt asks to change them.

## 9. Reference User Journey

- 1. Open the Generator Studio at /generate.
- 2. Choose input: Wireframe, Code, Prompt, or Combine.
- 3. Optionally set pageName (default Home) and sectionName (default Custom).
- 4. Submit. The Node API runs analysis + generation.
- 5. API writes a section document and N element documents, then returns generated JSX (or a component file path) plus IDs.

- 6. Frontend loads /preview/Home, dispatches fetchElementsByIds, and renders the section.
- 7. User may edit an element in a side panel; preview updates from store data, not from re-generating JSX.

## 10. System Architecture

| Component              | Runtime                | Responsibility                                                          |
|------------------------|------------------------|-------------------------------------------------------------------------|
| Generator Studio UI    | React                  | Capture wireframe, code, prompt; show job status and preview            |
| Preview shell          | React + Redux          | Provide cms.allSections and cms.allSectionsCss; mount generated section |
| Generate API           | Node.js                | Orchestrate parse → model → validate → persist → return artefact        |
| Vision / layout parser | Node.js + optional LLM | Turn image or JSX into a layout IR (intermediate representation)        |
| Code synthesiser       | Node.js + optional LLM | Emit React component that obeys the contract in Section 12              |
| Section store          | MongoDB / JSON         | CRUD for section metadata                                               |
| Element store          | MongoDB / JSON         | CRUD for CMS fields, including Cards.loop                               |
| Static storage         | Local disk             | Wireframes and placeholder images served under /storage                 |

### 10.1 Suggested intermediate representation (IR)

```
{
  "sectionType": "split-hero",
  "platform": "Website",
  "layout": {
    "direction": "row",
    "breakpoint": "md",
    "left": { "role": "media", "element": "heroImage" },
    "right": ["brandBadge", "headlineMain", "headlineSub", "description", "statBadges", "ctaButton"]
  },
  "theme": { "accent": "red-500", "surface": "white", "text": "gray-800" },
  "elements": [ /* see Appendix B */ ]
}
```

## 11. Domain Model and Data Contracts

The data shapes below are the source of truth for persistence and for the generated React bindings. Field names should be kept so judges can inspect documents quickly. Values are fictional.

### 11.1 Identifiers

- sectionId: 10-digit string unique within the project (example 1000000001).

- fieldId: 10-digit string unique within the project (example 2000000001).
- Nested card fields use fieldId1, fieldId2, ... on each loop item.
- pageName: human page key used in Redux, example Home.
- Do not expose real ObjectIds, cloud URLs, or git commit hashes in demos or repos.

## 11.2 Section document

| Field                   | Type          | Meaning                                          |
|-------------------------|---------------|--------------------------------------------------|
| sectionName             | string        | Logical type, e.g. Custom, Hero, Features        |
| sectionId               | string        | Stable ID referenced by elements                 |
| variations              | string/number | How many layout variants exist                   |
| path                    | string        | Virtual component path, not a real client folder |
| sectionStatus           | string        | Pending   Approved   Rejected                    |
| wireframes              | string        | Relative path to the source wireframe, if any    |
| platform                | string        | Website for this topic                           |
| pageName                | string        | Page the section belongs to                      |
| isGenerated             | boolean       | true when produced by the generator              |
| cardGridColumns         | number        | Default 3 for stats/cards                        |
| cardLayoutMode          | string        | grid   list                                      |
| sectionTextMode         | string        | auto   light   dark                              |
| sectionColor / paddings | string        | Optional theme tokens applied on the wrapper     |

## 11.3 Element document

| Field       | Type   | Meaning                                            |
|-------------|--------|----------------------------------------------------|
| sectionId   | string | Parent section                                     |
| elementName | string | Stable semantic name: heroImage, headlineMain, ... |
| fieldId     | string | DOM id and Redux key                               |
| content     | string | Default / current CMS value                        |
| contentType | enum   | Image   Text   Textfield   Button   Cards          |

|             |               |                                         |
|-------------|---------------|-----------------------------------------|
| css         | string   null | Inline cssText overlay for this fieldId |
| loop        | array         | Required when contentType is Cards      |
| projectName | string        | Use sample-brand only                   |
| pageName    | string        | Must match section.pageName             |

## 11.4 Cards loop item

```
{
  "field1": "1000+",
  "fieldType1": "Text",
  "fieldId1": "3000000001",
  "field2": "Community<br />Members",
  "fieldType2": "Text",
  "fieldId2": "3000000002"
}
```

## 11.5 Runtime Redux shape

```
state.cms.allSections[pageName][fieldId] = contentStringOrLoopArray
state.cms.allSectionsCss[pageName][fieldId] = cssTextString
state.cms.sectionNames = { [sectionId]: sectionName }
```

# 12. Generated Component Contract

Every generated section MUST follow these rules. Judges will open the file and the DOM.

## 12.1 Mandatory rules

- R1. Declare a const ids object mapping semantic names to fieldId strings.
- R2. Accept pageName as a prop with a sensible default (Home).
- R3. On mount, dispatch fetchElementsByIds with every fieldId used in the tree, including nested card field IDs.
- R4. Read live values from state.cms.allSections[pageName].
- R5. Every editable node has id={ids.something} or id={item.fieldIdN}.
- R6. Text nodes use dangerouslySetInnerHTML with a default fallback: data?.[id] || "DEFAULT".
- R7. Images use a getImage helper, VITE\_STORAGE\_URL prefix, blob: passthrough, and onError placeholder.
- R8. Buttons use PrimeReact Button (or documented fallback) with label from CMS and an onClick stub.
- R9. Repeating items render from a loop array with a DEFAULT\_\* constant when CMS data is missing or the wrong length.
- R10. Apply allSectionsCss to matching DOM ids after cssData changes.
- R11. Use Tailwind for layout. Desktop: two columns. Mobile: stacked. Include a max-width container.
- R12. Add class dynamicStyle on text/button nodes and dynamicStyle2 on images so a future editor can hook them.
- R13. Do not embed real secrets, real bucket URLs, or real customer identifiers.



- R14. Export default the section component.

## 12.2 Accessibility and quality rules

- Images have meaningful alt text derived from the prompt or a generic description.
- CTA has aria-label.
- Colour contrast: body text gray-500+ on white is acceptable; do not put gray-400 on white for long copy.
- Do not leave empty id attributes on nested card fields (the sample in Appendix C includes ids on both stat value and label).

## 12.3 What generated code must NOT do

- Hard-code production storage hosts.
- Import internal proprietary helpers that students cannot implement (replace with local utils).
- Skip Redux and only render defaults.
- Use Math.random() for field IDs on every render (IDs must be stable and persisted).

# 13. Functional Requirements

## 13.1 Generator Studio

| ID     | Pri    | Requirement                                                          |
|--------|--------|----------------------------------------------------------------------|
| FR-G01 | Must   | Provide a form to upload a wireframe image.                          |
| FR-G02 | Must   | Provide a textarea to paste existing React/JSX code.                 |
| FR-G03 | Must   | Provide a textarea for a natural-language prompt.                    |
| FR-G04 | Must   | Allow selecting one mode or combining two+ inputs in a single job.   |
| FR-G05 | Must   | Show generation progress and surface API errors in plain language.   |
| FR-G06 | Must   | After success, show generated JSX (read-only) and a link to preview. |
| FR-G07 | Should | Let the user set pageName, sectionName, and accent colour.           |
| FR-G08 | Should | Keep the last 5 jobs in a history list (in-memory is enough).        |
| FR-G09 | Could  | Download a zip: Component.jsx + section.json + elements.json.        |

## 13.2 Generation engine

| ID     | Pri  | Requirement                                                                                                                                            |
|--------|------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| FR-E01 | Must | Produce unique 10-digit fieldIds for new elements and persist them.                                                                                    |
| FR-E02 | Must | Map detected regions to the reference element set for the hero: herolImage, brandBadge, headlineMain, headlineSub, description, statBadges, ctaButton. |

|        |        |                                                                                               |
|--------|--------|-----------------------------------------------------------------------------------------------|
| FR-E03 | Must   | Create Cards.loop with exactly the count requested (default 3) and unique nested field IDs.   |
| FR-E04 | Must   | Emit JSX that compiles in the preview app without manual edits.                               |
| FR-E05 | Must   | Include default fallback copy for every text/button/image.                                    |
| FR-E06 | Should | If code input already contains an ids map, reuse those IDs (stable regeneration).             |
| FR-E07 | Should | Validate IR before synthesis: missing CTA or missing heading is a warning, not a silent drop. |
| FR-E08 | Could  | Return a confidence score per element when the source is a wireframe.                         |

### 13.3 Preview and CMS runtime

| ID     | Pri    | Requirement                                                                                 |
|--------|--------|---------------------------------------------------------------------------------------------|
| FR-P01 | Must   | Render the generated section inside a preview route.                                        |
| FR-P02 | Must   | Hydrate Redux from GET /elements?pageName=Home (or equivalent).                             |
| FR-P03 | Must   | Changing an element via API or a small editor updates the preview without regenerating JSX. |
| FR-P04 | Must   | Broken image paths fall back to the placeholder.                                            |
| FR-P05 | Should | Responsive preview: toggle mobile (~375px) and desktop (~1280px) widths.                    |
| FR-P06 | Should | Apply per-element css strings from the store onto DOM ids.                                  |
| FR-P07 | Could  | Side-by-side wireframe vs live preview slider.                                              |

### 13.4 Persistence APIs

| ID     | Pri    | Requirement                                                          |
|--------|--------|----------------------------------------------------------------------|
| FR-A01 | Must   | POST /api/generate accepts wireframe file and/or code and/or prompt. |
| FR-A02 | Must   | GET /api/sections and GET /api/sections/:sectionId.                  |
| FR-A03 | Must   | GET /api/elements?sectionId= or ?pageName=                           |
| FR-A04 | Must   | PATCH /api/elements/:fieldId to update content and css.              |
| FR-A05 | Should | POST /api/sections/:sectionId/regenerate for variation 2.            |

|        |        |                                  |
|--------|--------|----------------------------------|
| FR-A06 | Should | Health endpoint GET /api/health. |
|--------|--------|----------------------------------|

## 14. API Specification (minimum)

### 14.1 POST /api/generate

| Part        | Required         | Description                          |
|-------------|------------------|--------------------------------------|
| mode        | Yes              | wireframe   code   prompt   combined |
| prompt      | If mode needs it | Natural language instructions        |
| code        | If mode needs it | Existing React/JSX as text           |
| wireframe   | If mode needs it | Image file                           |
| pageName    | No               | Default Home                         |
| sectionName | No               | Default Custom                       |

POST /api/generate  
Content-Type: multipart/form-data

Sample success response:

```
{
  "ok": true,
  "sectionId": "1000000001",
  "pageName": "Home",
  "componentFile": "src/sections/generated/HeroSection.jsx",
  "elementIds": ["2000000001", "2000000002", "2000000003"],
  "warnings": ["Subheading inferred with low confidence"],
  "ir": { "sectionType": "split-hero" }
}
```

### 14.2 GET /api/elements

Returns the array used to fill Redux. Nested Cards must include loop. Text content may contain simple HTML (span, br, strong) because the React contract uses dangerouslySetInnerHTML.

### 14.3 Error model

```
{
  "ok": false,
  "error": {
    "code": "INVALID_INPUT",
    "message": "At least one of wireframe, code, or prompt is required."
  }
}
```

Use HTTP 400 for validation, 413 for oversized images (recommend max 8 MB), 422 for model/parse failure, 500 for unexpected errors.

## 15. Non-Functional Requirements

| ID     | Area            | Requirement                                                                                  |
|--------|-----------------|----------------------------------------------------------------------------------------------|
| NFR-01 | Performance     | Preview first paint of a generated section under 2s on local after data is cached.           |
| NFR-02 | Performance     | Generate API should respond within 60s; show a spinner if the LLM is slower.                 |
| NFR-03 | Reliability     | If the LLM fails, return 422 with a message; do not write partial IDs without a section row. |
| NFR-04 | Security        | No secrets in git. Sanitize HTML from CMS to a small tag allow-list if time allows.          |
| NFR-05 | Security        | Do not execute user-pasted code on the server (parse as text / AST only).                    |
| NFR-06 | Privacy         | Use only fictional brand copy. Strip any real names found in pasted code before saving.      |
| NFR-07 | Usability       | Empty states and inline validation on the generate form.                                     |
| NFR-08 | Maintainability | README with setup, .env.example, and a 3-step demo script for judges.                        |
| NFR-09 | Compatibility   | Chrome or Edge latest. Node 18+ recommended (document if constrained).                       |
| NFR-10 | Accessibility   | Keyboard reachable generate form; preview CTA focus ring visible.                            |

**Note:** dangerouslySetInnerHTML is part of the CMS contract. Teams should still avoid introducing script tags. A simple allow-list (b, i, br, span, strong, em) is a judged plus.

## 16. Sample Scenario (for the event brief)

A fitness brand needs a Home page hero. Design has provided a split layout: athlete image on the left; on the right a red eyebrow, large headline, supporting line, short paragraph, three statistics, and a primary button. Content should be CMS-editable after launch.

### 16.1 Sample prompt (give this to students)

Create a full-viewport website hero for a fitness brand named Pulse Fit. Layout: two columns on desktop, stacked on mobile. Left: product / athlete image. Right: uppercase red badge, extra-bold headline, muted subheading, body paragraph, three stat cards in a row (value + label), and a red filled CTA button. Use white

background, red accent bars on far left and far right for large screens. All text, image, stats, and button must be CMS-bound with stable IDs and defaults.

## 16.2 Sample default copy

| Element       | Default content                                                                                    |
|---------------|----------------------------------------------------------------------------------------------------|
| brandBadge    | PULSE FIT                                                                                          |
| headlineMain  | CHALLENGE YOUR LIMITS                                                                              |
| headlineSub   | Be a part of the tribe that's limitless.                                                           |
| description   | Join trainer-led workout sessions designed to kickstart your fitness journey, at your convenience. |
| stat 1        | 1000+ / Community Members                                                                          |
| stat 2        | 40+ / Fitness Programmes                                                                           |
| stat 3        | 150+ / Fitness Channels                                                                            |
| ctaButton     | FIND A WORKOUT                                                                                     |
| heroImage alt | Athlete performing a dumbbell exercise                                                             |

## 17. Deliverables

- 1. Source repository (Git) with React app and Node API.
- 2. README: setup, how to run, how to demo each input mode, known limitations.
- 3. .env.example with dummy values only.
- 4. At least one generated section checked in under src/sections/generated/ as a reference output.
- 5. Seed JSON for one section and its elements (sanitized sample data).
- 6. Short demo script (5–8 minutes) covering wireframe, prompt, and code inputs if implemented.
- 7. Optional: screenshots or a screen recording of preview vs wireframe.

Do not submit: production dumps, real customer JSON, API keys, or internal commit hashes.

## 18. Suggested Timeline

| Hours     | Focus                                                                         |
|-----------|-------------------------------------------------------------------------------|
| 0.0 – 0.5 | Read this document. Split roles: API, generator UI, preview/contract.         |
| 0.5 – 2.0 | Scaffold React + Node. Redux cms slice. Dummy HeroSection from Appendix C.    |
| 2.0 – 4.0 | POST /api/generate for prompt mode. Persist section + elements. Wire preview. |

|           |                                                             |
|-----------|-------------------------------------------------------------|
| 4.0 – 6.0 | Wireframe upload + IR. Improve layout fidelity. Cards loop. |
| 6.0 – 7.0 | Code-input path OR content editor + PATCH element.          |
| 7.0 – 8.0 | README, seed data, dry-run demo, remove secrets.            |

## 19. Evaluation Rubric

| Criterion               | Points | What judges look for                                                               |
|-------------------------|--------|------------------------------------------------------------------------------------|
| CMS contract compliance | 25     | ids map, fetch on mount, fallbacks, fieldIds in DOM, Cards loop, css overlay hook  |
| Input coverage          | 20     | Working prompt mode (required). Wireframe and/or code mode. Combined inputs bonus. |
| Layout fidelity         | 15     | Split hero, stats row, CTA, responsive stack, accent treatment                     |
| Backend quality         | 15     | Generate + CRUD APIs, persistence, validation, no secrets                          |
| Code quality            | 10     | Readable React/Node, README, .env.example, no dead proprietary imports             |
| UX of the studio        | 10     | Clear form, errors, preview, optional editor                                       |
| Innovation / stretch    | 5      | IR, sanitiser, variations, zip export, vision labels                               |

### 19.1 Disqualification / major penalties

- Submitting real client data, secrets, or production URLs: 0 for security and possible disqualification.
- Frontend not in React.js or backend not in Node.js: topic requirement failed (cap at 20 unless exception granted).
- Preview only shows static HTML with no store/API: cap at 40.
- Plagiarism of another team's generator: disqualification.

## 20. Rules and Constraints

- Teams of 2–4 unless the organiser states otherwise.
- Open-source libraries are allowed. Document them in README.
- LLM APIs are allowed. Teams must run with a missing key in a degraded way (clear error), not a crash.
- Internet may be limited; a prompt-template fallback that still emits the contract is valued.
- All demo content must be fictional (Pulse Fit / sample-brand).
- Mentors may clarify requirements but will not write the generator for a team.

## 21. Setup Expectations for Organisers

- Provide this Word document plus a sample wireframe PNG (boxes labelled Hero image / Badge / Headline / Stats / CTA).
- Provide Appendix C as a starting preview component if you want a baseline compile.
- Provide a blank React+Node starter only if time is short; otherwise scaffolding is part of the challenge.
- Judges should have Node and a browser; MongoDB Atlas or local Mongo is optional if JSON store is accepted.

## 22. Risks and Mitigations

| Risk                            | Mitigation                                                                        |
|---------------------------------|-----------------------------------------------------------------------------------|
| LLM output is invalid JSX       | Validate with a parser; retry once; fall back to a template filled from IR.       |
| IDs collide across jobs         | Allocate IDs in the API (central counter or nanoid numeric), never in the LLM.    |
| Students paste proprietary code | Brief them to use Appendix C only; add a pre-submit grep for forbidden hostnames. |
| Images 404                      | Ship a default placeholder under /storage/default/images/.                        |
| PrimeReact install issues       | Allow native button with the same id/class/label contract.                        |

## 23. Glossary

| Term          | Meaning in this hackathon                                           |
|---------------|---------------------------------------------------------------------|
| Section       | A full-width page block (e.g. hero) with metadata and many elements |
| Element       | One CMS-editable field (text, image, button) with a fieldId         |
| Cards / loop  | A repeating group stored as an array of field pairs                 |
| IR            | Intermediate representation of layout before JSX is written         |
| dynamicStyle  | Marker class for text/button nodes hooked by an editor              |
| dynamicStyle2 | Marker class for image nodes                                        |
| Variation     | An alternate layout of the same sectionId family                    |
| Preview       | Runtime that hydrates CMS data into the generated React tree        |

## 24. Acceptance Checklist (students self-score before demo)

- [ ] Prompt mode generates a split hero that compiles.

- [ ] At least one of: wireframe mode or code mode works end-to-end.
- [ ] Section JSON and element JSON are stored and retrievable.
- [ ] DOM shows the expected ids (inspect heroImage, headlineMain, ctaButton).
- [ ] Changing headline via PATCH updates preview without code edit.
- [ ] Three stat cards render from loop with nested field IDs.
- [ ] Image fallback works when src is empty.
- [ ] README lists exact run commands.
- [ ] No API keys or real client names in the repo.
- [ ] Mobile stack and desktop split both demonstrated.

## Appendix A — Sample Section Document (sanitized)

```
{
  "_id": { "$oid": "aaaaaaaaaaaaaaaaaaaaaa" },
  "sectionName": "Custom",
  "sectionId": "1000000001",
  "variations": "2",
  "path": "/client/sample-brand/Custom/CustomHero",
  "sectionStatus": "Pending",
  "wireframes": "sample/wireframes/hero-section.png",
  "platform": "Website",
  "createdAt": { "$date": "2026-07-03T12:27:09.463Z" },
  "updatedAt": { "$date": "2026-07-03T12:27:09.463Z" },
  "pageId": { "$oid": "bbbbbbbbbbbbbbbbbbbb" },
  "pageName": "Home",
  "isGenerated": true,
  "layoutPlacementId": "ly-000000000000-sample",
  "isBuild": true,
  "cardClassName": "",
  "cardCssText": "",
  "cardGridColumns": 3,
  "cardLayoutMode": "grid",
  "cardListBodyGapPx": 15,
  "sectionClassName": "",
  "sectionColor": "",
  "sectionPaddingBottom": "",
  "sectionPaddingLeft": "",
  "sectionPaddingRight": "",
  "sectionPaddingTop": "",
  "sectionTextMode": "auto",
  "tableClassName": "",
  "tableCssText": ""
}
```

## Appendix B — Sample Element Documents (sanitized)

```
[
  {
    "_id": { "$oid": "cccccccccccccccccccc" },
    "sectionId": "1000000001",
    "elementName": "heroImage",
    "fieldId": "2000000001",
    "content": "default/images/hero-placeholder.jpg",
    "contentType": "Image",
    "section": "Custom",
    "projectName": "sample-brand",
  }
]
```



```

"pageId": { "$oid": "bbbbbbbbbbbbbbbbbbbbbb" },
"pageName": "Home",
"isCustom": true,
"css": null
},
{
  "elementName": "brandBadge",
  "fieldId": "2000000002",
  "content": "PULSE FIT",
  "contentType": "Text"
},
{
  "elementName": "headlineMain",
  "fieldId": "2000000003",
  "content": "CHALLENGE YOUR LIMITS",
  "contentType": "Text",
  "css": "font-weight: bold; text-align: left;"
},
{
  "elementName": "headlineSub",
  "fieldId": "2000000004",
  "content": "Be a part of the tribe that's limitless.",
  "contentType": "Text"
},
{
  "elementName": "description",
  "fieldId": "2000000005",
  "content": "Join trainer-led workout sessions designed to kickstart your fitness journey, at your convenience.",
  "contentType": "Textfield"
},
{
  "elementName": "statBadges",
  "fieldId": "2000000006",
  "content": "",
  "contentType": "Cards",
  "loop": [
    {
      "field1": "100+",
      "fieldType1": "Text",
      "fieldId1": "3000000001",
      "field2": "Community<br />Members",
      "fieldType2": "Text",
      "fieldId2": "3000000002"
    },
    {
      "field1": "40+",
      "fieldType1": "Text",
      "fieldId1": "3000000003",
      "field2": "Fitness<br />Programmes",
      "fieldType2": "Text",
      "fieldId2": "3000000004"
    },
    {
      "field1": "150+",
      "fieldType1": "Text",
      "fieldId1": "3000000005",
      "field2": "Fitness<br />Channels",
      "fieldType2": "Text",
      "fieldId2": "3000000006"
    }
  ]
},
{
  "elementName": "ctaButton",

```

```

    "fieldId": "2000000007",
    "content": "FIND A WORKOUT",
    "contentType": "Button"
  }
]

```

## Appendix C — Sample Generated React Section (sanitized)

```

import React, { useEffect, useRef } from "react";
import { useSelector, useDispatch } from "react-redux";
import { Button } from "primereact/button";
import { fetchElementsByIds } from "../../redux";
import { getSectionTextContrastClass } from "../../utils/sectionContrast";

const ids = {
  heroImage: "2000000001",
  brandBadge: "2000000002",
  headlineMain: "2000000003",
  headlineSub: "2000000004",
  description: "2000000005",
  statBadges: "2000000006",
  ctaButton: "2000000007",
};

const DEFAULT_STAT_CARDS = [
  {
    field1: "100+",
    fieldType1: "Text",
    fieldId1: "3000000001",
    field2: "Community<br />Members",
    fieldType2: "Text",
    fieldId2: "3000000002",
  },
  {
    field1: "40+",
    fieldType1: "Text",
    fieldId1: "3000000003",
    field2: "Fitness<br />Programmes",
    fieldType2: "Text",
    fieldId2: "3000000004",
  },
  {
    field1: "150+",
    fieldType1: "Text",
    fieldId1: "3000000005",
    field2: "Fitness<br />Channels",
    fieldType2: "Text",
    fieldId2: "3000000006",
  },
];

function getImage(image) {
  if (!image) return `${import.meta.env.VITE_STORAGE_URL}default/images/hero-placeholder.jpg`;
  if (image.includes("blob:")) return image;
  return `${import.meta.env.VITE_STORAGE_URL}${image}`;
}

function errorImage(event) {
  event.target.src = `${import.meta.env.VITE_STORAGE_URL}default/images/hero-placeholder.jpg`;
}

```

```

const HeroSection = ({ pageName = "Home" }) => {
  const dispatch = useDispatch();
  const data = useSelector((state) => state.cms?.allSections?.[pageName]);
  const cssData = useSelector((state) => state.cms?.allSectionsCss?.[pageName]);
  const rootRef = useRef(null);
  const textContrastClass = getSectionTextContrastClass(undefined, undefined);

  const statBadgesArr =
    Array.isArray(data?.[ids.statBadges]) && data[ids.statBadges].length === 3
      ? data[ids.statBadges]
      : DEFAULT_STAT_CARDS;

  useEffect(() => {
    dispatch(
      fetchElementsByIds({
        elementIds: [
          ids.heroImage,
          ids.brandBadge,
          ids.headlineMain,
          ids.headlineSub,
          ids.description,
          ids.statBadges,
          "3000000001",
          "3000000002",
          "3000000003",
          "3000000004",
          "3000000005",
          "3000000006",
          ids.ctaButton,
        ],
        pageName,
      })
    );
  }, [dispatch, pageName]);

  useEffect(() => {
    Object.values(ids).forEach((id) => {
      if (id && cssData?.[id]) {
        const el = document.getElementById(id);
        if (el) el.style.cssText = cssData[id];
      }
    });
  }, [cssData]);

  const handleCtaClick = () => {
    alert("Find a workout clicked!");
  };

  return (
    <div ref={rootRef} className={`relative min-h-screen w-full flex items-center justify-center overflow-hidden bg-background-color ${textContrastClass}`}>
      <div className="hidden md:block absolute left-0 top-0 h-full w-8 bg-red-500 z-0" />
      <div className="hidden md:block absolute right-0 top-0 h-full w-8 bg-red-500 z-0" />
      <main className="relative z-10 w-full max-w-[1920px] mx-auto px-0 md:px-12 flex items-center justify-center min-h-screen">
        <section className="w-full flex flex-col md:flex-row bg-white overflow-hidden">
          <div className="md:w-1/2 w-full flex items-center justify-center min-h-[400px] md:min-h-[680px] py-8 md:py-16 bg-white">
            <img id={ids.heroImage} className="dynamicStyle2 max-w-full max-h-[600px] h-auto object-contain mx-auto" src={data?.[ids.heroImage] ? getImage(data[ids.heroImage]) : `${import.meta.env.VITE_STORAGE_URL}default/images/hero-placeholder.jpg`} alt="Athlete performing a dumbbell exercise" onError={errorImage} />
          </div>
          <div className="md:w-1/2 w-full flex flex-col justify-center px-6 md:px-16 py-12 md:py-16 space-y-8 bg-white">

```

```

    <span id={ids.brandBadge} className="dynamicStyle uppercase text-red-500 font-bold text-lg"
    dangerouslySetInnerHTML={{ __html: data?.[ids.brandBadge] || "PULSE FIT" }} />
    <h1 id={ids.headlineMain} className="dynamicStyle text-4xl md:text-5xl font-extrabold
    tracking-tight text-gray-800 leading-tight" dangerouslySetInnerHTML={{ __html: data?.[ids.headlineMain]
    || "CHALLENGE YOUR LIMITS" }} />
    <h2 id={ids.headlineSub} className="dynamicStyle text-lg md:text-xl font-medium text-gray-
    500 tracking-wide" dangerouslySetInnerHTML={{ __html: data?.[ids.headlineSub] || "Be a part of the tribe
    that's limitless." }} />
    <p id={ids.description} className="dynamicStyle text-gray-500 text-base md:text-lg max-w-xl
    leading-relaxed" dangerouslySetInnerHTML={{ __html: data?.[ids.description] || "Join trainer-led workout
    sessions designed to kickstart your fitness journey." }} />
    <div id={ids.statBadges} className="dynamicStyle flex flex-col md:flex-row md:justify-
    between md:gap-x-8 gap-y-8 pt-2">
      {statBadgesArr.map((item) => (
        <div key={item.fieldId1} className="flex-1 flex flex-col items-center justify-center">
          <div id={item.fieldId1} className="text-3xl md:text-4xl font-extrabold text-gray-800"
          dangerouslySetInnerHTML={{ __html: data?.[item.fieldId1] || item.field1 }} />
          <div id={item.fieldId2} className="mt-1 text-gray-500 text-base"
          dangerouslySetInnerHTML={{ __html: data?.[item.fieldId2] || item.field2 }} />
        </div>
      ))}
    </div>
    <Button id={ids.ctaButton} className="dynamicStyle w-full md:w-auto bg-red-500 hover:bg-red-
    600 text-white font-semibold rounded" severity="danger" aria-label="Find a workout"
    label={data?.[ids.ctaButton] || "FIND A WORKOUT"} onClick={handleCtaClick} />
  </div>
</section>
</main>
</div>
);
};
export default HeroSection;

```

## Appendix D — Mapping: Wireframe regions to elements

| Wireframe region (typical)  | elementName  | contentType | DOM notes                               |
|-----------------------------|--------------|-------------|-----------------------------------------|
| Left large media box        | heroImage    | Image       | id + dynamicStyle2 + getImage + onError |
| Small uppercase label       | brandBadge   | Text        | span, HTML allowed                      |
| Largest heading             | headlineMain | Text        | h1                                      |
| Secondary heading / tagline | headlineSub  | Text        | h2                                      |
| Paragraph under tagline     | description  | Textfield   | p                                       |
| Row of 3 metric blocks      | statBadges   | Cards       | container id; children use fieldId1/2   |
| Primary action              | ctaButton    | Button      | PrimeReact Button label bound to CMS    |

## Appendix E — Sample judge demo script (8 minutes)

- 1. Show README and .env.example (30s). Start API + React.
- 2. Prompt mode: paste the sample prompt. Show IR or logs. Open preview (2 min).
- 3. Inspect DOM for ids. PATCH headlineMain to “TRAIN WITHOUT LIMITS” and refresh preview (1.5 min).
- 4. Wireframe or code mode: generate again. Point out reused or new IDs (2 min).
- 5. Responsive toggle and one known limitation (1 min).
- 6. Questions (remaining time).

## Appendix F — Data sanitisation policy used in this brief

- Real project / brand names → sample-brand / Pulse Fit.
- Real MongoDB ObjectIds → placeholder 24-character ids.
- Real 10-digit production field IDs → 1000... / 2000... / 3000... ranges.
- Real storage and CDN URLs → VITE\_STORAGE\_URL only.
- Git commit hashes, deploy flags, and internal request filenames → omitted or replaced with sample/wireframes/hero-section.png.
- Internal package paths and proprietary helper names → short local equivalents (fetchElementsByIds).
- Employee or machine file paths → not included.

# Problem Statement ID – PS8: Binding Automation

**Note:** All IDs, ObjectIds, project names, and internal paths must be fictional or sanitized. No real credentials.

## 1. Purpose

This topic defines how a backend must bind form input fields to a dynamically chosen function (stored in MongoDB) and return a validated output object.

## 2. Reference Implementation (what the hackathon is based on)

Your backend already exposes a dynamic binding API surface in ``formsApi.ts`` and ``functionAPI`` service logic.

### 2.1 Dynamic routes (from ``formsApi.ts``)

| Route                              | Purpose                                      | Expected binding behaviour                                                                           |
|------------------------------------|----------------------------------------------|------------------------------------------------------------------------------------------------------|
| <code>/forms/function/*</code>     | Execute a stored custom function by name     | Load function document, build execution context from request + form payload, return computed output. |
| <code>/forms/createfunction</code> | Register (create) a custom function document | Persist function definition and its schemas (input/output contract).                                 |
| <code>/forms/getfunction</code>    | Fetch one function document                  | Used by admin/studio to show function contract.                                                      |
| <code>/forms/getAllfunction</code> | Fetch list of function documents             | Used to populate a function selector in UI.                                                          |

### 2.2 Execution model (from ``functionAPI`` service)

- The service finds the function document by `projectName` and function name (from the route parameter).
- The function is stored as a string; the service currently evaluates it and calls it with the service instance as context.
- The function reads/writes MongoDB models using the same request context (e.g., `request.body` identifiers).

**Note:** Security requirement for teams: avoid using raw ``eval`` for student implementations; use a safe function registry or sandboxed VM (scored as a +).

## 3. Data Contracts (MongoDB schemas)

There are two core document types: Form Schemas (fields) and Custom Functions (binding logic + input/output contract).

### 3.1 Form schema document (example based on `hrx-new\_forms\_schemas.json`)

| Field property | Type          | Meaning / binding impact                                             |
|----------------|---------------|----------------------------------------------------------------------|
| field          | string        | Stable field key used as payload key and UI label reference.         |
| dataType       | string enum   | Type hints for validation (e.g., String, Number).                    |
| label          | string        | Display label in UI and used in error messages.                      |
| defaultValue   | string        | Default payload value when missing.                                  |
| inputType      | string        | UI input widget type (input, textarea, phone, url, multiline, etc.). |
| validation     | boolean       | Enable/disable validation checks.                                    |
| mandatoryField | boolean       | If true, binding engine must reject empty payloads.                  |
| mappedColumn   | string   null | Optional mapping to a database column for persistence workflows.     |
| mappedTableRef | string   null | Optional mapping to a related table for joins/population workflows.  |
| multipleSelect | boolean       | If true, expected payload is an array and output must reflect that.  |

### 3.2 Custom function document (example based on `assetflow\_custom\_functions.json`)

| Field property                | Type                         | Meaning / binding impact                                                       |
|-------------------------------|------------------------------|--------------------------------------------------------------------------------|
| name                          | string                       | Function selector key (used in /forms/function/:name route).                   |
| function                      | string (code) OR template id | Logic to compute output from input payload / DB context.                       |
| payloadStructure.bodyType     | string                       | Describes the content type (e.g., form-data) expected by function execution.   |
| payloadStructure.requestBody  | object                       | Which request body keys are used (e.g., an _id reference to load DB records).  |
| payloadStructure.bodyParams[] | array                        | List of input keys and whether they are active/enabled in the binding request. |
| expectedResponseSchema        | object                       | Defines output fields and their types for validation/mapping.                  |

**Note:** Teams may simplify stored `function` representation. What matters is that the system can: (1) know required input keys, (2) compute output, and (3) validate output fields against expectedResponseSchema.

## 4. Binding Automation Requirements

### 4.1 Core goal

Given a form payload (input fields) and a selected custom function name, the backend must execute the function and return an output object that matches expectedResponseSchema.

### 4.2 Functional requirements

- A. Function registry and selection: The API must allow registering functions with a unique `name` per `projectName`. The API must allow listing and fetching function documents to support a UI selector.
- B. Input payload binding: When executing `/forms/function/\*`, the backend must load the function document and read required input keys from `payloadStructure` and the incoming request payload/body. Mandatory inputs defined by the function contract must be validated before execution. If form schema defines defaults for missing fields, binding must merge defaults before executing the function. If multipleSelect is true for a field, binding must enforce array type and validate each element if validation is enabled.
- C. Execution and output mapping: The function execution must return an object (or record) that contains the expectedResponseSchema fields. The backend must validate output fields by type (string/number/boolean/date as needed) and either coerce or reject invalid values based on a defined rule. The backend must return the output in the standard response wrapper: `res.success.success('Request Acknowledged', data)` equivalent (teams can define their own wrapper but should keep `data` key).
- D. Error handling: 404 Function not found if the requested name does not exist. 400 Validation error when required inputs are missing or wrong type. 422 Execution/parse failure when function logic fails or cannot read required DB records. 500 Unexpected error with safe logs only (no secrets in response).

### 4.3 Security and quality requirements (important)

- Do not run untrusted code directly with raw `eval` in student implementations. Use a safe registry of allowed templates or a sandbox runner.
- Do not allow functions to access environment secrets or filesystem unless explicitly allowed by judges.
- Validate and sanitize any user-provided strings before using them in DB queries (use parameterized queries where possible).
- Keep function templates versioned; function output schema changes must bump version in the function document.

## 5. Example (sanitized) contracts and payload

### 5.1 Sample form payload

```
{
  "projectName": "sample-forms",
  "formSchemaName": "accounts",
  "payload": {
    "accountname": "Example Corp",
    "phone": "9999999999",
    "website": "https://example.com",
    "industry": "Technology"
  }
}
```



```
}
```

## 5.2 Sample function execution request

```
{
  "route": "POST /forms/function/RequestInformationAssetDetail",
  "body": {
    "projectName": "sample-forms",
    "_id": "<placeholder-id>"
  }
}
```

## 5.3 Sample validated output shape

```
{
  "assetType": "Laptop",
  "model": "Model-A",
  "quantity": 2,
  "requestType": "Purchase",
  "priority": "High",
  "deadline": "2026-12-31",
  "estimatedCost": 150000,
  "currency": "INR",
  "department": 12,
  "deliveryLocation": "Warehouse-1",
  "reason": "Business need",
  "businessJustification": "Demo only",
  "createdOn": "2026-08-11"
}
```

## 6. Deliverables (student teams)

- 1. Node.js + Express (or equivalent) backend exposing the required `/forms/\*` endpoints.
- 2. MongoDB collections (or JSON file store) for: form schemas and custom functions.
- 3. An execution engine that binds form payload -> function input -> validated output.
- 4. At least 1 registered custom function and 1 demo payload that passes end-to-end.
- 5. README: run steps, how to register a function, and how to test `/forms/function/:name`.

## 7. Evaluation rubric (100 points)

| Criterion                   | Points | Judging focus                                                                    |
|-----------------------------|--------|----------------------------------------------------------------------------------|
| Binding contract compliance | 30     | mandatory inputs, defaults, multipleSelect handling, and output schema matching. |
| API correctness             | 20     | Function register/list/get + execute endpoints behave as specified.              |
| Validation quality          | 20     | Type checks, coercion rules, and consistent error codes/messages.                |
| Security posture            | 15     | No raw eval in final submission; safe sandbox/registry.                          |

|                     |    |                                                          |
|---------------------|----|----------------------------------------------------------|
| Code quality + docs | 15 | Readable code, clear contracts, README, and demo script. |
|---------------------|----|----------------------------------------------------------|

## Appendix A: Sanitized sample function document

```
[
  {
    "name": "RequestInformationAssetDetail",
    "function": "<template-or-code-string>",
    "payloadStructure": {
      "bodyType": "form-data",
      "requestBody": { "_id": "<placeholder-id>" },
      "bodyParams": [
        { "key": "projectName", "value": "sample-forms", "isActive": false },
        { "key": "_id", "value": "<placeholder-id>", "isActive": true }
      ]
    },
    "expectedResponseSchema": {
      "rootType": "object",
      "fields": [
        { "field": "assetType", "type": "string", "isActive": true },
        { "field": "quantity", "type": "number", "isActive": true }
      ]
    }
  }
]
```

**Note:** Appendix examples are sanitized. Teams must keep their submission fictional (no secrets, no real customer identifiers).

# Problem Statement ID – PS9: Dynamic Function Creation

**Note:** All project names, ObjectIds, collection names, and function bodies in this document are fictional sample data. Do not use real client identifiers, production dumps, or API keys in submissions.

## 1. Document Control

| Field           | Value                                     |
|-----------------|-------------------------------------------|
| Mandatory stack | Node.js backend + MongoDB persistence     |
| Classification  | Internal hackathon use - sample data only |

### 1.1 Purpose

This document defines a hackathon challenge where teams build a system that reads project context (existing form schemas, table relationships, and a natural-language requirement) and automatically detects the required business logic. The system must then generate and persist an appropriate custom function document - including function code (or template), input payload contract, and expected output schema - compatible with the binding automation model described in Topic 2.

## 2. Executive Summary

In a low-code / form-driven platform, developers repeatedly write similar MongoDB lookup and aggregation functions: fetch a record by id, join related schemas, map fields to a response DTO, and expose the result through POST /forms/function/:functionName.

The hackathon challenge is to automate this step. Given a projectName and a requirement such as 'When user submits an asset request id, return asset type, model, quantity, priority, and deadline', the system must:

- 1. Load project context from MongoDB (forms schemas, existing functions, optional sample records).
- 2. Detect intent: read, create, update, delete, aggregate, validate, or transform.
- 3. Select referenced schemas and fields from mappedTableRef / field definitions.
- 4. Generate a custom function document with name, function body, payloadStructure, and expectedResponseSchema.
- 5. Persist via POST /forms/createfunction (or equivalent) and prove execution via POST /forms/function/:name.

## 3. Problem Statement

Manual function authoring is slow and inconsistent because:

- Business analysts describe intent in plain language, but developers must translate it into Mongo queries.
- Input keys (\_id, projectName, filters) and output field names differ across teams.
- payloadStructure and expectedResponseSchema are often missing or out of sync with the function body.
- Existing form schemas already describe tables and fields, but that metadata is not reused intelligently.

Students must close this gap with a Function Creation Engine that produces binding-ready artefacts, not just raw JavaScript snippets.

## 4. Hackathon Objectives

### 4.1 Primary objectives (must achieve)

- Accept projectName + natural-language requirement as input.
- Read forms schema documents for that project from MongoDB.
- Detect business logic type and referenced schemas/fields.
- Generate and persist a complete custom function document (name, function, payloadStructure, expectedResponseSchema).
- Execute the generated function successfully through the dynamic function API.

### 4.2 Secondary objectives (strongly recommended)

- Support requirement + example payload (few-shot) to improve field mapping accuracy.
- Detect joins across mappedTableRef relationships and generate multi-schema lookups.
- Return generationMeta with intent, confidence, and referenced schemas for audit.
- Reject ambiguous requirements with clarifying questions or a structured warning list.

### 4.3 Stretch objectives (bonus)

- Auto-create or extend a forms schema when the requirement references fields not yet defined.
- Version functions: regenerate v2 while preserving name with version suffix or metadata.version.
- Generate unit-test fixtures from expectedResponseSchema.
- Simple admin UI to review, approve, and publish generated functions before activation.

## 5. Scope

### 5.1 In scope

- Node.js API for function generation, registration, listing, and execution.
- MongoDB collections: forms\_schemas (or {project}\_forms\_schemas) and custom\_functions (or {project}\_custom\_functions).
- Intent detection from requirement text + project schema context.
- Generation of payloadStructure.bodyParams and expectedResponseSchema.fields.
- At least three supported intents: read\_single, read\_with\_join, aggregate\_summary.

### 5.2 Out of scope

- Real production projects, real customer data, or copying proprietary function bodies verbatim.
- Training a foundation model from scratch.
- Full CI/CD deploy pipeline for generated code.
- Unrestricted arbitrary code execution without validation (see security section).

## 6. System Architecture

| Component              | Runtime                 | Responsibility                                                     |
|------------------------|-------------------------|--------------------------------------------------------------------|
| Function Studio API    | Node.js                 | Accept requirement, trigger generation pipeline, return preview    |
| Project Context Loader | Node.js + MongoDB       | Load schemas, existing functions, optional seed records            |
| Intent Detector        | Node.js + LLM/rules     | Classify business logic: read, join, aggregate, mutate, validate   |
| Schema Planner         | Node.js                 | Pick schemas, fields, joins, filters from forms schema metadata    |
| Function Synthesizer   | Node.js + LLM/templates | Emit async function body using getSchemaModel pattern              |
| Contract Builder       | Node.js                 | Build payloadStructure and expectedResponseSchema from planned I/O |
| Function Registry      | MongoDB                 | Persist custom function documents                                  |
| Function Executor      | Node.js                 | POST /forms/function/:name - same contract as Topic 2              |

### 6.1 End-to-end pipeline

Requirement + projectName → Load project context → Intent detection → Schema + field planning → Generate function body + I/O contracts → Validate against schema metadata → Persist to custom\_functions collection → Execute via POST /forms/function/:generatedName

### 6.2 Reference routes (compatible with existing forms API pattern)

| Route                   | Method   | Purpose                                                                  |
|-------------------------|----------|--------------------------------------------------------------------------|
| /forms/generateFunction | POST     | Main entry: requirement in, generated function document out              |
| /forms/createfunction   | POST     | Persist approved function (may be called internally by generateFunction) |
| /forms/getfunction      | POST     | Fetch one function by name                                               |
| /forms/getAllfunction   | POST     | List functions for projectName                                           |
| /forms/function/:name   | POST     | Execute generated function with payload                                  |
| /forms/schemas          | GET/POST | Optional: list or register forms schema documents                        |

## 7. Project Context Model

Project context is everything the generator needs to infer correct business logic without guessing table names.

### 7.1 Context inputs

| Context source     | Collection / artefact                    | Used for                                      |
|--------------------|------------------------------------------|-----------------------------------------------|
| projectName        | Request body                             | Select per-project Mongo collections          |
| Forms schemas      | {project}_forms_schemas or forms_schemas | Field names, data types, mappedTableRef joins |
| Existing functions | {project}_custom_functions               | Avoid duplicate names; reuse patterns         |
| Sample records     | Dynamic per schemaName                   | Optional few-shot field mapping validation    |
| Requirement text   | Request body.requirement                 | Primary intent and business rules             |
| Example payload    | Request body.examplePayload              | Optional input key hints                      |
| Example output     | Request body.exampleOutput               | Optional output field hints                   |

### 7.2 Forms schema field contract (from shared sample)

| Property       | Generation use                                                             |
|----------------|----------------------------------------------------------------------------|
| field          | Mongo field key and output property source                                 |
| dataType       | Output type in expectedResponseSchema (String -> string, Number -> number) |
| label          | Human-readable description in response schema                              |
| inputType      | Hint for input validation in payloadStructure                              |
| mandatoryField | Mark bodyParam as required when used as input                              |
| mappedTableRef | Trigger join planning to related schema                                    |
| mappedColumn   | Target column when cross-schema mapping exists                             |

### 7.3 Custom function document contract (output of this topic)

| Field    | Required | Description                                                  |
|----------|----------|--------------------------------------------------------------|
| name     | Yes      | Unique function name within project (PascalCase recommended) |
| function | Yes      | Async function body as string OR template reference          |

|                        |        |                                                                   |
|------------------------|--------|-------------------------------------------------------------------|
| payloadStructure       | Yes    | bodyType, requestBody, bodyParams[] with isActive flags           |
| expectedResponseSchema | Yes    | rootType + fields[] with type and isActive                        |
| isActive / isDeleted   | Yes    | Lifecycle flags                                                   |
| generationMeta         | Should | intent, confidence, referencedSchemas, sourceRequirement, version |

## 8. Business Logic Intent Detection

The core differentiator of this topic is detecting what logic is required before writing code. Teams may use rule-based keyword matching, LLM classification, or a hybrid approach.

### 8.1 Supported intent catalogue (minimum)

| Intent code       | Trigger phrases (examples)                        | Generated behaviour                                |
|-------------------|---------------------------------------------------|----------------------------------------------------|
| read_single       | get by id, fetch detail, show record              | findOne on primary schema using _id from payload   |
| read_with_join    | include related, with department, asset type name | findOne + lookup on mappedTableRef schema          |
| aggregate_summary | count, total, summary, list grouped               | aggregate or count with mapped output fields       |
| validate_record   | check if exists, validate before submit           | existence check returning boolean + message        |
| transform_map     | map fields, rename, format response               | read + field rename to expectedResponseSchema keys |

### 8.2 Intent detection output (internal IR)

```
{
  "intent": "read_with_join",
  "primarySchema": "asset_requests",
  "joins": [
    { "fromField": "request_id", "targetSchema": "assets", "targetField": "_id" },
    { "fromField": "asset_type_id", "targetSchema": "asset_types", "targetField": "_id" }
  ],
  "inputKeys": ["_id", "projectName"],
  "outputFields": [
    { "name": "assetType", "source": "asset_types.asset_name", "type": "string" },
    { "name": "quantity", "source": "asset_requests.quantity", "type": "number" }
  ],
  "confidence": 0.82,
  "warnings": ["deadline mapped from required_by_date"]
}
```

### 8.3 Ambiguity handling

- If requirement references a schema not found in project context, return 422 with suggested schema names.
- If multiple schemas match (e.g. 'request'), return top candidates with confidence scores.
- If output fields are underspecified, infer from schema fields mentioned in requirement or return defaults from primary schema.

## 9. Functional Requirements

### 9.1 Function generation API

| ID      | Priority | Requirement                                                                                        |
|---------|----------|----------------------------------------------------------------------------------------------------|
| FR-GF01 | Must     | POST /forms/generateFunction accepts projectName and requirement (string).                         |
| FR-GF02 | Must     | Load all active forms schemas for projectName before generation.                                   |
| FR-GF03 | Must     | Detect intent and referenced schemas from requirement + context.                                   |
| FR-GF04 | Must     | Generate unique function name derived from intent and primary schema (e.g. GetAssetRequestDetail). |
| FR-GF05 | Must     | Generate function body using getSchemaModel + findOne/find/aggregate pattern.                      |
| FR-GF06 | Must     | Generate payloadStructure with bodyParams for each required input key.                             |
| FR-GF07 | Must     | Generate expectedResponseSchema.fields aligned with return object keys and types.                  |
| FR-GF08 | Must     | Persist function via createfunction flow unless previewOnly=true.                                  |
| FR-GF09 | Must     | Return generation preview: function document + generationMeta + warnings.                          |
| FR-GF10 | Should   | Accept examplePayload and exampleOutput to refine mapping.                                         |
| FR-GF11 | Should   | Reject duplicate function names with 409 or auto-suffix (v2).                                      |
| FR-GF12 | Could    | Support regenerateFunction to update an existing function by name.                                 |

### 9.2 Generated function body rules

- R1. Function must be async and return a plain object matching expectedResponseSchema.
- R2. Use getSchemaModel(schemaName) helper pattern to resolve dynamic Mongoose models from forms schema.
- R3. Read input from request.body (at minimum projectName and primary key such as \_id).
- R4. Throw clear errors when schema or record not found (Topic 2 executor maps these to HTTP errors).
- R5. Map Mongo field names (snake\_case) to response keys (camelCase) when requirement implies DTO naming.
- R6. Include only fields marked isActive:true in expectedResponseSchema.



- R7. Do not hard-code real project names or real ObjectIds in generated code.

### 9.3 Schema generation (optional extension)

If teams implement auto-schema creation, new forms schema documents must follow the same field array structure as Appendix A. Generated schemas must include `schemaName`, `projectName`, `formIdRef`, `fields[]`, `isActive`, `isDeleted`. New `schemaName` must be unique within project. Each field must define `field`, `dataType`, `label`, `type`, `inputType`, `validation`, `mandatoryField`. When requirement introduces a relationship, set `mappedTableRef` on the foreign-key field.

### 9.4 Validation before persist

- Validate generated JSON against an internal JSON Schema for custom function documents.
- Parse function body with a JS parser (e.g. `acorn`) to ensure syntactically valid async function.
- Verify every output field in `expectedResponseSchema` appears in a `return {}` statement or template map.
- Verify every `isActive` `bodyParam` has a corresponding reference in function body or documented as injected.

## 10. API Specification

### 10.1 POST /forms/generateFunction

POST /forms/generateFunction  
Content-Type: application/json

```
{
  "projectName": "sample-flow",
  "requirement": "When user provides an account id, return account name, phone, website, and industry.",
  "examplePayload": { "_id": "<placeholder-id>" },
  "exampleOutput": { "accountName": "Example Corp", "phone": 999999999 },
  "previewOnly": false,
  "preferredFunctionName": "GetAccountSummaryById"
}
```

Sample success response:

```
{
  "ok": true,
  "data": {
    "functionDocument": { /* see Appendix B */ },
    "generationMeta": {
      "intent": "read_single",
      "referencedSchemas": ["accounts"],
      "confidence": 0.86,
      "warnings": []
    },
    "executeRoute": "POST /forms/function/GetAccountSummaryById"
  }
}
```

### 10.2 POST /forms/createfunction (persist)

```
{
  "projectName": "sample-flow",
  "name": "GetAccountSummaryById",
  "function": "async function GetAccountSummaryById() { ... }",
  "payloadStructure": { /* generated */ },
  "expectedResponseSchema": { /* generated */ }
```

```
}
```

### 10.3 POST /forms/function/:name (prove it works)

```
POST /forms/function/GetAccountSummaryById
```

```
{
  "projectName": "sample-flow",
  "_id": "<placeholder-id>"
}
```

## 11. Sample Hackathon Scenario

Organisers provide a seeded MongoDB project sample-flow with schemas: accounts, asset\_requests, assets, asset\_types (all fictional). Teams receive the following requirement: When a user submits an asset request id, return asset type name, model, quantity, request type, priority, deadline, estimated cost, currency, department, delivery location, reason, business justification, and created date.

Expected system behaviour: 1. Detect intent read\_with\_join across asset\_requests, assets, asset\_types. 2. Generate function name such as GetAssetRequestInformation. 3. Build payloadStructure with active \_id and projectName params. 4. Build expectedResponseSchema with 13 output fields and correct types. 5. Persist and execute; return mapped DTO for a seed record id.

### 11.1 Simpler starter scenario (for 8-hour sprint)

Requirement: 'Given an account id, return account name and phone.' Single schema accounts only. Intent read\_single. Minimum viable demo.

## 12. Non-Functional Requirements

| ID     | Area            | Requirement                                                                  |
|--------|-----------------|------------------------------------------------------------------------------|
| NFR-01 | Performance     | Generation completes within 90 seconds including LLM call.                   |
| NFR-02 | Reliability     | Partial generation must not persist broken function documents.               |
| NFR-03 | Security        | No raw eval of unvalidated code in production path; validate before persist. |
| NFR-04 | Security        | LLM prompts must not include secrets; API keys in .env only.                 |
| NFR-05 | Auditability    | generationMeta stored with every generated function.                         |
| NFR-06 | Maintainability | README with seed script, sample requirement, and demo curl commands.         |
| NFR-07 | Compatibility   | Node.js 18+ and MongoDB 5+ or Atlas free tier.                               |

## 13. Deliverables

- 1. Node.js repository with generateFunction pipeline and MongoDB integration.
- 2. Seed data: at least 2 forms schemas + sample records (fictional).
- 3. At least 2 successfully generated and executed functions demonstrated live.
- 4. JSON export of one generated function document (Appendix B format).
- 5. README: architecture, intent list, setup, demo script.
- 6. .env.example with dummy values only.

## 14. Evaluation Rubric (100 points)

| Criterion                 | Points | Judging focus                                                   |
|---------------------------|--------|-----------------------------------------------------------------|
| Intent detection accuracy | 25     | Correct intent, schemas, joins inferred from requirement        |
| Contract completeness     | 25     | payloadStructure + expectedResponseSchema match function output |
| Execution success         | 20     | POST /forms/function/:name returns valid data for seed record   |
| Context utilisation       | 15     | Uses forms schema metadata (fields, mappedTableRef, dataType)   |
| Code quality + security   | 10     | Validation before persist, no secrets, readable pipeline        |
| Innovation                | 5      | Ambiguity handling, schema auto-create, approval UI, versioning |

**Important:** A team that returns hard-coded JavaScript without detecting intent or building schemas cannot score above 35 points.

## 15. Suggested Timeline

| Hours | Focus                                                            |
|-------|------------------------------------------------------------------|
| 0 - 1 | Seed MongoDB, implement context loader and list schemas API      |
| 1 - 3 | Rule-based or LLM intent detector + IR output                    |
| 3 - 5 | Function synthesizer (template for read_single)                  |
| 5 - 6 | Contract builder for payloadStructure and expectedResponseSchema |
| 6 - 7 | Persist + execute via /forms/function/:name                      |
| 7 - 8 | Second scenario (join), README, demo                             |

## 16. Rules and Constraints

- Backend must be Node.js. Persistence must be MongoDB (local or Atlas).
- Generated functions must follow the custom function document contract from Topic 2.
- Teams may use any LLM API or offline rule engine; document the approach.
- All demo data must be fictional.
- Copying production function bodies from organiser samples is not allowed.

## Appendix A - Sample Forms Schema Document (sanitized)

```
{
  "_id": { "$oid": "aaaaaaaaaaaaaaaaaaaaaa" },
  "projectName": "sample-flow",
  "schemaName": "accounts",
  "formIdRef": "sample-flow_forms",
  "isActive": true,
  "isDeleted": false,
  "fields": [
    {
      "field": "accountname",
      "dataType": "String",
      "label": "Account Name",
      "type": "text",
      "inputType": "input",
      "validation": true,
      "mandatoryField": false,
      "mappedColumn": null,
      "mappedTableRef": null
    },
    {
      "field": "phone",
      "dataType": "Number",
      "label": "Phone",
      "type": "text",
      "inputType": "phone",
      "validation": true,
      "mandatoryField": false
    }
  ]
}
```

## Appendix B - Sample Generated Custom Function Document (sanitized)

```
{
  "name": "GetAccountSummaryById",
  "function": "async function GetAccountSummaryById() { /* generated logic */ }",
  "isActive": true,
  "isDeleted": false,
  "payloadStructure": {
    "bodyType": "json",
    "requestBody": { "_id": "<placeholder-id>", "projectName": "sample-flow" },
    "bodyParams": [
      { "key": "projectName", "value": "sample-flow", "isActive": true },
      { "key": "_id", "value": "<placeholder-id>", "isActive": true }
    ]
  },
  "expectedResponseSchema": {
    "rootType": "object",
  }
}
```

```

    "fields": [
      { "field": "accountName", "type": "string", "description": "Resolved account name", "isActive": true },
      { "field": "phone", "type": "number", "description": "Primary phone", "isActive": true },
      { "field": "recordCount", "type": "number", "description": "Related records found", "isActive": true }
    ],
    "generationMeta": {
      "sourceRequirement": "Fetch account summary for a given record id",
      "detectedIntent": "read_single_with_aggregation",
      "referencedSchemas": ["accounts"],
      "confidence": 0.86,
      "version": 1
    }
  }
}

```

## Appendix C - Sample Generated Function Body (sanitized)

```

async function GetAccountSummaryById() {
  const getSchemaModel = async (schemaName) => {
    const schema = await this.FormsSchemaModel(request.body.projectName).findOne({
      schemaName,
      isActive: true,
      isDeleted: false,
    });
    if (!schema) throw new Error(schemaName + " schema not found");
    return this.getModel(schemaName, request.body.projectName, schema.fields);
  };

  const AccountModel = await getSchemaModel("accounts");
  const account = await AccountModel.findOne(
    { _id: Types.ObjectId(request.body._id) },
    { accountname: 1, phone: 1, website: 1, industry: 1 }
  ).lean();

  if (!account) throw new Error("Account not found");

  return {
    accountName: account.accountname || null,
    phone: account.phone || null,
    website: account.website || null,
    industry: account.industry || null,
  };
}

```

## Appendix D - Mapping: Requirement phrases to generation artefacts

| Requirement signal          | Detected intent | Generated artefact                                |
|-----------------------------|-----------------|---------------------------------------------------|
| by id / using _id           | read_single     | bodyParam _id; findOne on primary schema          |
| include / with related X    | read_with_join  | Second getSchemaModel; lookup by foreign key      |
| return name, phone, website | transform_map   | expectedResponseSchema fields with camelCase keys |

|                          |                   |                                                 |
|--------------------------|-------------------|-------------------------------------------------|
| count / total / how many | aggregate_summary | countDocuments or aggregate pipeline            |
| accounts table / schema  | schema reference  | primarySchema = accounts from forms schema list |

## Appendix E - Judge demo script (8 minutes)

- 1. Show seeded schemas in MongoDB for sample-flow (1 min).
- 2. POST generateFunction with account summary requirement (2 min).
- 3. Review generationMeta: intent, schemas, confidence (1 min).
- 4. Execute POST /forms/function/:name with seed \_id (2 min).
- 5. Show payloadStructure and expectedResponseSchema match response (1 min).
- 6. Optional: second requirement with join (remaining time).

## Appendix F - Sanitisation policy

- Real project names (hrx-new, assetflow) replaced with sample-flow / sample-brand.
- Real ObjectIds replaced with placeholder tokens or fictional 24-char ids.
- Production function source not included; only structural patterns documented.
- No internal file paths, commit hashes, or credentials in student submissions.

# Problem Statement ID – PS10: Business-Context API Test Generation

**Note:** All project names, ObjectIds, route paths, and function names in this document are fictional sample data. No real credentials, production identifiers, or proprietary code may appear in student submissions.

## 1. Document Control

| Field           | Value                                                        |
|-----------------|--------------------------------------------------------------|
| Mandatory stack | Node.js + TypeScript, Playwright for API testing, MongoDB    |
| Optional extras | Allure / HTML report, MongoDB test fixtures via seed scripts |

### 1.1 Purpose

Teams build a system that reads project context (forms schemas from MongoDB, existing custom function documents, and an optional business requirement description) and automatically identifies relevant test cases. It then generates runnable Playwright TypeScript API test suites and verifies them against a live Node.js backend.

## 2. Executive Summary

The dynamic forms API provides a uniform surface: POST /forms/formGet/:schema, POST /forms/formCreate/:schema, POST /forms/formUpdate/:schema, POST /forms/formDelete/:schema, POST /forms/formBulkupload/:schema, POST /forms/function/:name, POST /forms/createfunction, POST /forms/getAllfunction, and POST /forms/query/:schema. These routes are driven entirely by schema metadata stored in MongoDB, which means test cases can also be derived from that same metadata automatically.

The challenge: given a projectName and a business requirement, the system must: 1. Load forms schema fields for each schema in the project. 2. Identify relevant test scenarios: CRUD, validation, edge-case, and function-execution tests. 3. Generate Playwright TypeScript spec files for each identified scenario. 4. Run tests against a live or seeded instance and surface a pass/fail report.

## 3. Problem Statement

Manual test authoring is repetitive and schema-dependent:

- Every schema has CRUD routes with the same structure; writing 5 tests per schema by hand is error-prone.
- mandatoryField=true on a schema field should always produce a 'missing field' negative test, but that is often skipped.
- mappedTableRef fields require population/join tests that are complex to write by hand.
- Custom functions have an expectedResponseSchema; its fields should map directly to Playwright expect() matchers.
- Business requirements add domain-specific rules (e.g. 'phone must be 10 digits') that generate extra assertions.

Students must close this gap: a generation engine that reads schema metadata and requirements, derives a test catalogue, and emits Playwright spec files that a judge can run with `npx playwright test`.

## 4. Hackathon Objectives

### 4.1 Primary objectives (must achieve)

- Accept projectName + optional requirement description as input.
- Load all active forms schemas for the project from MongoDB.
- Identify and catalogue test cases per schema: at minimum CRUD happy-path and mandatory-field negative tests.
- Generate Playwright TypeScript spec files with test.describe blocks, beforeAll seed fixtures, and expect assertions.
- Generated tests must pass (green) against the reference API with seed data.

### 4.2 Secondary objectives (strongly recommended)

- Generate function-execution tests from custom function documents (expectedResponseSchema -> matchers).
- Generate function-registry tests (create, duplicate-reject, list, get).
- Derive validation tests from field-level rules: dataType Number -> NaN rejection, url inputType -> bad-URL rejection.
- Generate soft-delete and isDeleted filter tests when schema has isDeleted field.
- Produce an HTML or Allure report automatically after test run.

### 4.3 Stretch objectives (bonus)

- Detect business rules from requirement text (e.g. 'phone must be 10 digits') and generate custom assertion logic.
- Generate bulk-upload tests with CSV/JSON fixture files built from schema fields.
- Generate query/aggregation tests from existing queryResponse configurations.
- Publish test results as a JSON summary consumable by a CI system.
- Admin UI to browse identified test catalogue and selectively generate/run specs.

## 5. Scope

### 5.1 In scope

- Node.js + TypeScript test-generation engine backed by MongoDB schema context.
- Playwright API tests (no browser UI, HTTP-level only via request.newContext()).
- Test catalogue for: formGet, formCreate, formUpdate, formDelete, formBulkupload, function/:name, createfunction, getAllfunction.
- Seed fixture management: beforeAll / afterAll helpers that create and clean up test data.

### 5.2 Out of scope

- Browser / Playwright UI tests (e2e visual testing is a separate topic).
- Real client schemas, credentials, or production data.
- Load / performance testing.



- Authentication middleware tests (middleware is out of scope for unauthenticated form routes used in demo).

## 6. Mandatory Technology Stack

| Layer           | Required                            | Notes                                                    |
|-----------------|-------------------------------------|----------------------------------------------------------|
| Test runtime    | Playwright @playwright/test         | API tests via request.newContext() - no browser needed   |
| Language        | TypeScript (strict mode)            | tsconfig.json must include strict: true                  |
| Test generator  | Node.js (any framework or plain TS) | Reads MongoDB context, emits .spec.ts files              |
| Database        | MongoDB                             | Hosts forms_schemas and custom_functions for the project |
| Schema reader   | Mongoose or native MongoDB driver   | To query FormsSchemaModel and customFunctionModel        |
| Config          | playwright.config.ts                | testDir, timeout, reporter (html + list minimum)         |
| Env             | .env / .env.example                 | API_BASE_URL, MONGODB_URI (no real values committed)     |
| Package manager | npm or pnpm                         | package.json with test script wired to `playwright test` |

### 6.1 Folder structure (recommended)

```

hackathon-test-gen/
  generator/
    contextLoader.ts  <- Node.js TS engine that writes spec files
    catalogBuilder.ts <- loads MongoDB schemas + function docs
    specWriter.ts     <- identifies test cases from schema metadata
    index.ts          <- emits Playwright .spec.ts files
    index.ts          <- CLI entry point
  tests/              <- generated spec files (committed after generation)
    forms/            <- per-schema CRUD specs
    functions/        <- custom function + registry specs
  fixtures/           <- seed JSON for beforeAll helpers
  playwright.config.ts
  tsconfig.json
  package.json
  .env.example
  README.md

```

## 7. Project Context Model

### 7.1 Context inputs

| Source | Used for |
|--------|----------|
|--------|----------|

|                            |                                                                                              |
|----------------------------|----------------------------------------------------------------------------------------------|
| Forms schema documents     | Derive field names, types, mandatory flags, mappedTableRef for CRUD test data and assertions |
| Custom function documents  | Derive function name, input bodyParams, expectedResponseSchema for function tests            |
| Business requirement text  | Derive domain-specific assertions (value ranges, formats, cross-field rules)                 |
| Example payload (optional) | Use as valid fixture in happy-path tests                                                     |
| Example output (optional)  | Use in toMatchObject expectations                                                            |

## 7.2 From forms schema: field → test case mapping

| Schema field property  | Derived test cases                                                          |
|------------------------|-----------------------------------------------------------------------------|
| mandatoryField: true   | TC negative: omit this field in create payload → expect error response      |
| dataType: Number       | TC negative: send string for this field → expect error or coercion          |
| inputType: url         | TC negative: send malformed URL → expect rejection                          |
| inputType: phone       | TC positive: send valid 10-digit number; TC negative: send letters          |
| mappedTableRef != null | TC: create record, populate join via formGet, verify nested object returned |
| multipleSelect: true   | TC: send array value; TC negative: send scalar                              |
| defaultValue != empty  | TC: omit field, verify default value appears in fetched record              |
| enum.length > 0        | TC: send value outside enum → expect rejection                              |

## 7.3 From custom function documents: contract → test mapping

| Function document field                        | Derived test cases                                                      |
|------------------------------------------------|-------------------------------------------------------------------------|
| payloadStructure.bodyParams[].isActive = true  | TC: active param present (happy path); TC: active param missing (error) |
| payloadStructure.bodyParams[].isActive = false | TC: param absent, function should still succeed                         |
| expectedResponseSchema.fields[].type           | TC: assert response field matches declared type                         |
| name                                           | TC: POST /forms/function/:name succeeds; unknown name returns 404       |
| isActive: false                                | TC: inactive function should not execute (if system supports it)        |

## 8. Test Case Catalogue (generated per schema)

The generation engine must produce at minimum the following test IDs for every schema. Replace <SCHEMA> with the actual schemaName (e.g. accounts).

### 8.1 CRUD test cases

| ID         | Category            | Description                                                     | Expected result                           |
|------------|---------------------|-----------------------------------------------------------------|-------------------------------------------|
| TC-CRUD-01 | Create – happy path | POST /forms/formCreate/<SCHEMA> with all valid fields           | 200, record returned with _id             |
| TC-CRUD-02 | Read – by id        | POST /forms/formGet/<SCHEMA> with valid _id                     | 200, record matches created payload       |
| TC-CRUD-03 | Read – list         | POST /forms/formGet/<SCHEMA> without _id (paginated)            | 200, records array + count                |
| TC-CRUD-04 | Read – search       | POST /forms/formGet/<SCHEMA> with searchBy matching known value | 200, filtered results contain match       |
| TC-CRUD-05 | Update              | POST /forms/formUpdate/<SCHEMA> with _id and changed field      | 200, updated value reflected              |
| TC-CRUD-06 | Delete (soft)       | POST /forms/formDelete/<SCHEMA> with _id                        | 200, isDeleted set to true                |
| TC-CRUD-07 | Deleted filter      | POST /forms/formGet/<SCHEMA> with isDeleted: false              | 200, deleted record not in list           |
| TC-CRUD-08 | Create – mandatory  | POST /forms/formCreate/<SCHEMA> omitting each mandatoryField    | Error flag / non-200 / validation message |
| TC-CRUD-09 | Create – wrong type | POST /forms/formCreate/<SCHEMA> with Number field as string     | Error or coercion (assert intent)         |
| TC-CRUD-10 | Missing schema      | POST /forms/formGet/nonexistent                                 | Error: schema not found                   |

### 8.2 Bulk upload test cases

| ID         | Category            | Description                                                    | Expected result       |
|------------|---------------------|----------------------------------------------------------------|-----------------------|
| TC-BULK-01 | Valid file          | POST /forms/formBulkupload/<SCHEMA> with correct CSV/JSON file | 200, records inserted |
| TC-BULK-02 | Invalid file format | POST /forms/formBulkupload/<SCHEMA> with wrong MIME type       | Error response        |

|            |              |                                              |                                                   |
|------------|--------------|----------------------------------------------|---------------------------------------------------|
| TC-BULK-03 | Partial rows | File with some rows missing mandatory fields | Partial success or full rejection (assert intent) |
|------------|--------------|----------------------------------------------|---------------------------------------------------|

### 8.3 Custom function execution test cases

| ID         | Category              | Description                                           | Expected result                                    |
|------------|-----------------------|-------------------------------------------------------|----------------------------------------------------|
| TC-FUNC-01 | Execute – happy path  | POST /forms/function/:name with all active bodyParams | 200, response matches expectedResponseSchema types |
| TC-FUNC-02 | Execute – missing key | POST /forms/function/:name with active param omitted  | Error: application-level failure                   |
| TC-FUNC-03 | Unknown name          | POST /forms/function/NonExistent                      | 404 or equivalent error                            |
| TC-FUNC-04 | Type assertion        | Each field in expectedResponseSchema asserted by type | All field types match declared types               |
| TC-FUNC-05 | Project mismatch      | POST /forms/function/:name with wrong projectName     | Error or empty result                              |

### 8.4 Function registry test cases

| ID        | Category             | Description                                                 | Expected result                         |
|-----------|----------------------|-------------------------------------------------------------|-----------------------------------------|
| TC-REG-01 | Create function      | POST /forms/createfunction with valid name + function body  | 200, document persisted                 |
| TC-REG-02 | Duplicate name       | POST /forms/createfunction with duplicate name              | 409 / error: already exists             |
| TC-REG-03 | Get function         | POST /forms/getfunction with known fName                    | 200, correct document returned          |
| TC-REG-04 | List all             | POST /forms/getAllfunction for projectName                  | 200, array includes registered function |
| TC-REG-05 | Execute after create | Create function then immediately execute via function/:name | 200, function result returned           |

### 8.5 Business-rule test cases (requirement-derived)

When a requirement text is supplied, the engine must extract domain assertions and add them. Examples for a requirement: 'Phone must be 10 digits and account name must not be empty':

| ID | Derived from requirement | Assertion |
|----|--------------------------|-----------|
|----|--------------------------|-----------|

|           |                                |                                                 |
|-----------|--------------------------------|-------------------------------------------------|
| TC-BIZ-01 | phone must be 10 digits        | Send 9-digit phone → expect error               |
| TC-BIZ-02 | phone must be 10 digits        | Send 10-digit phone → expect success            |
| TC-BIZ-03 | account name must not be empty | Send empty string accountname → expect error    |
| TC-BIZ-04 | account name must not be empty | Send whitespace-only accountname → expect error |

## 9. Functional Requirements

### 9.1 Context loader

| ID      | Priority | Requirement                                                                                                  |
|---------|----------|--------------------------------------------------------------------------------------------------------------|
| FR-TG01 | Must     | Connect to MongoDB, list active schemas for projectName.                                                     |
| FR-TG02 | Must     | Load fields[] for each schema including dataType, mandatoryField, inputType, mappedTableRef, multipleSelect. |
| FR-TG03 | Must     | Load custom function documents including payloadStructure and expectedResponseSchema.                        |
| FR-TG04 | Should   | Accept optional business requirement text as additional context input.                                       |
| FR-TG05 | Could    | Load a sample record from each schema to build realistic fixture values.                                     |

### 9.2 Test catalogue builder

| ID      | Priority | Requirement                                                                  |
|---------|----------|------------------------------------------------------------------------------|
| FR-TG06 | Must     | Identify all test cases in Section 8.1 for each active schema.               |
| FR-TG07 | Must     | Identify at least one negative test per mandatoryField=true field.           |
| FR-TG08 | Must     | Identify function tests (TC-FUNC-01 to TC-FUNC-05) for each custom function. |
| FR-TG09 | Should   | Identify business-rule tests from requirement text.                          |
| FR-TG10 | Should   | Return a test catalogue JSON before generating files (for preview/approval). |
| FR-TG11 | Could    | Detect mappedTableRef relationships and add join-population tests.           |

### 9.3 Spec file writer

| ID | Priority | Requirement |
|----|----------|-------------|
|----|----------|-------------|

|         |        |                                                                                          |
|---------|--------|------------------------------------------------------------------------------------------|
| FR-TG12 | Must   | Emit one .spec.ts file per schema under tests/forms/.                                    |
| FR-TG13 | Must   | Emit one .spec.ts file per function (or one combined) under tests/functions/.            |
| FR-TG14 | Must   | Each spec file uses Playwright test.describe, test, and expect. No other test framework. |
| FR-TG15 | Must   | beforeAll seeds required records; afterAll or test-scoped cleanup removes them.          |
| FR-TG16 | Must   | Use process.env.API_BASE_URL (fallback localhost) and process.env.PROJECT_NAME.          |
| FR-TG17 | Should | Add a test.step() label per logical assertion group.                                     |
| FR-TG18 | Could  | Emit an index spec that imports all others for a single npx playwright test run.         |

## 9.4 Test runner and reporting

| ID      | Priority | Requirement                                                                       |
|---------|----------|-----------------------------------------------------------------------------------|
| FR-TG19 | Must     | Generated tests run successfully with npx playwright test against seeded API.     |
| FR-TG20 | Must     | HTML report generated via playwright reporter html.                               |
| FR-TG21 | Should   | Exit code 0 on all-pass, non-zero on any failure.                                 |
| FR-TG22 | Could    | JSON summary written to test-results/summary.json with pass/fail counts per spec. |

# 10. Generator API Specification

## 10.1 POST /test-gen/generate

POST /test-gen/generate  
Content-Type: application/json

```
{
  "projectName": "sample-flow",
  "requirement": "Phone must be 10 digits. Account name must not be empty.",
  "examplePayload": { "accountname": "Example Corp", "phone": 9999999999 },
  "schemas": ["accounts"],
  "includeFunctions": true,
  "outputDir": "./tests",
  "previewOnly": false
}
```

Sample success response:

```
{
  "ok": true,
  "data": {
    "catalogue": [
      { "id": "TC-CRUD-01", "schema": "accounts", "category": "create-happy-path" },

```

```

    { "id": "TC-BIZ-01", "schema": "accounts", "category": "business-rule" }
  ],
  "filesWritten": ["tests/forms/accounts.spec.ts", "tests/functions/getAccountSummary.spec.ts"],
  "totalTests": 14
}
}

```

## 10.2 POST /test-gen/run

```

POST /test-gen/run
Content-Type: application/json

```

```

{ "outputDir": "./tests", "reportDir": "./playwright-report" }

```

Response includes passed, failed, skipped counts and a link to HTML report.

## 11. Sample Hackathon Scenario

Judges will seed a MongoDB project sample-flow with schemas: accounts (10 fields, 2 mandatoryField=true fields) and one custom function GetAccountSummaryByld (2 active bodyParams, 4 expectedResponseSchema fields). Teams receive: projectName: sample-flow, requirement: Phone must be exactly 10 digits. Account name must not be empty. Target schemas: [accounts], Target functions: [GetAccountSummaryByld].

Minimum expected output (to pass demo): 1. tests/forms/accounts.spec.ts – at least TC-CRUD-01 through TC-CRUD-08 + TC-BIZ-01 to TC-BIZ-04. 2. tests/functions/getAccountSummary.spec.ts – at least TC-FUNC-01 to TC-FUNC-04. 3. tests/functions/functionRegistry.spec.ts – at least TC-REG-01 to TC-REG-04. 4. npx playwright test exits 0 with all generated tests green.

## 12. Non-Functional Requirements

| ID     | Area            | Requirement                                                                         |
|--------|-----------------|-------------------------------------------------------------------------------------|
| NFR-01 | Performance     | Generation completes in under 60 seconds for up to 5 schemas.                       |
| NFR-02 | Reliability     | If MongoDB is unreachable, generator exits with a clear error message, not a crash. |
| NFR-03 | Idempotency     | Running generator twice must not create duplicate spec files or test IDs.           |
| NFR-04 | Security        | No secrets in generated spec files or committed test fixtures.                      |
| NFR-05 | Readability     | Generated TypeScript must pass tsc --noEmit without errors.                         |
| NFR-06 | Compatibility   | Node.js 18+, Playwright 1.40+, TypeScript 5+.                                       |
| NFR-07 | Maintainability | README must show how to add a new schema and re-run generation.                     |

## 13. Deliverables

- 1. Generator source under generator/ (Node.js TypeScript, documented).

- 2. Generated test specs under tests/ (committed as evidence of generation).
- 3. playwright.config.ts, tsconfig.json, package.json.
- 4. Seed script under fixtures/ to initialise sample-flow data.
- 5. HTML test report screenshot or live demo of npx playwright test passing.
- 6. README: setup, seed command, generate command, run command.
- 7. .env.example with dummy values.

## 14. Evaluation Rubric (100 points)

| Criterion                   | Points | Judging focus                                                        |
|-----------------------------|--------|----------------------------------------------------------------------|
| Test catalogue accuracy     | 25     | Correct test cases identified from schema metadata + requirement     |
| Playwright spec quality     | 25     | Correct assertions, describe structure, beforeAll fixtures, env vars |
| All-green test run          | 20     | npx playwright test passes against seeded API                        |
| Business-rule test coverage | 15     | Requirement-derived assertions present and correct                   |
| Code quality + types        | 10     | TypeScript strict, tsc clean, readable generator code                |
| Innovation / stretch        | 5      | Bulk tests, join tests, JSON summary, CI-ready output                |

**Important:** A team that hand-writes tests without a generation engine scores at most 35 points. The generator must demonstrably read MongoDB context and emit spec files.

## 15. Suggested Timeline

| Hours   | Focus                                                                       |
|---------|-----------------------------------------------------------------------------|
| 0 – 1   | Connect to MongoDB, implement context loader, list schemas + functions      |
| 1 – 2.5 | Catalogue builder: derive CRUD + mandatory-field test case list from schema |
| 2.5 – 4 | Spec writer: emit accounts.spec.ts with CRUD tests                          |
| 4 – 5   | Function test spec writer: getAccountSummary + functionRegistry             |
| 5 – 6   | Business-rule parser from requirement text                                  |
| 6 – 7   | Run npx playwright test, fix generated code until green                     |
| 7 – 8   | README, seed script, HTML report, demo dry-run                              |



## Appendix A – Sample Generated CRUD Spec (accounts)

```
// tests/forms/accounts.spec.ts
import { test, expect, request } from "@playwright/test";

const BASE = process.env.API_BASE_URL ?? "http://localhost:4000";
const PROJECT = "sample-flow";
const SCHEMA = "accounts";
let createdId: string;

test.describe("Accounts CRUD - generated from schema context", () => {

  test("TC-CRUD-01 | create a valid account record", async () => {
    const ctx = await request.newContext();
    const res = await ctx.post(`${BASE}/forms/formCreate/${SCHEMA}`, {
      data: { projectName: PROJECT, accountname: "Example Corp", phone: 9999999999 },
    });
    expect(res.status()).toBe(200);
    const body = await res.json();
    expect(body.data).toMatchObject({ accountname: "Example Corp" });
    createdId = body.data._id;
  });

  test("TC-CRUD-02 | fetch the created record by id", async () => {
    const ctx = await request.newContext();
    const res = await ctx.post(`${BASE}/forms/formGet/${SCHEMA}`, {
      data: { projectName: PROJECT, _id: createdId },
    });
    expect(res.status()).toBe(200);
    const body = await res.json();
    expect(body.data.records[0]._id).toBe(createdId);
  });

  test("TC-CRUD-03 | reject create with missing mandatory field", async () => {
    const ctx = await request.newContext();
    const res = await ctx.post(`${BASE}/forms/formCreate/${SCHEMA}`, {
      data: { projectName: PROJECT }, // missing accountname
    });
    const body = await res.json();
    expect(body.ok ?? body.success).toBeFalsy();
  });

  test("TC-CRUD-04 | soft-delete the record", async () => {
    const ctx = await request.newContext();
    const res = await ctx.post(`${BASE}/forms/formDelete/${SCHEMA}`, {
      data: { projectName: PROJECT, _id: createdId },
    });
    expect(res.status()).toBe(200);
  });

  test("TC-CRUD-05 | deleted record not returned in list", async () => {
    const ctx = await request.newContext();
    const res = await ctx.post(`${BASE}/forms/formGet/${SCHEMA}`, {
      data: { projectName: PROJECT, isDeleted: false },
    });
    const body = await res.json();
    const ids = (body.data.records ?? []).map((r: any) => r._id);
    expect(ids).not.toContain(createdId);
  });
});
```

## Appendix B – Sample Generated Function Execution Spec

```
// tests/functions/getAccountSummary.spec.ts
import { test, expect, request } from "@playwright/test";

const BASE    = process.env.API_BASE_URL ?? "http://localhost:4000";
const PROJECT = "sample-flow";
const FUNC    = "GetAccountSummaryById";
let seedId: string;

test.beforeAll(async () => {
  const ctx = await request.newContext();
  const res = await ctx.post(`${BASE}/forms/formCreate/accounts`, {
    data: { projectName: PROJECT, accountname: "Seed Corp", phone: 8888888888 },
  });
  const body = await res.json();
  seedId = body.data._id;
});

test.describe("Custom function execution - GetAccountSummaryById", () => {

  test("TC-FUNC-01 | execute function with valid _id", async () => {
    const ctx = await request.newContext();
    const res = await ctx.post(`${BASE}/forms/function/${FUNC}`, {
      data: { projectName: PROJECT, _id: seedId },
    });
    expect(res.status()).toBe(200);
    const body = await res.json();
    expect(body.data).toMatchObject({
      accountName: expect.any(String),
      phone: expect.any(Number),
    });
  });

  test("TC-FUNC-02 | return 404 for unknown function name", async () => {
    const ctx = await request.newContext();
    const res = await ctx.post(`${BASE}/forms/function/NonExistentFn`, {
      data: { projectName: PROJECT, _id: seedId },
    });
    expect(res.status()).toBe(404);
  });

  test("TC-FUNC-03 | return error for missing _id in payload", async () => {
    const ctx = await request.newContext();
    const res = await ctx.post(`${BASE}/forms/function/${FUNC}`, {
      data: { projectName: PROJECT },
    });
    const body = await res.json();
    expect(body.ok ?? body.success).toBeFalsy();
  });
});
```

## Appendix C – Sample Generated Function Registry Spec

```
// tests/functions/functionRegistry.spec.ts
import { test, expect, request } from "@playwright/test";

const BASE    = process.env.API_BASE_URL ?? "http://localhost:4000";
const PROJECT = "sample-flow";
const FN_NAME = "TestAutoFunction";
```

```

test.describe("Function registry - createfunction / getfunction / getAllfunction", () => {

  test("TC-REG-01 | register a new function", async () => {
    const ctx = await request.newContext();
    const res = await ctx.post(`${BASE}/forms/createfunction`, {
      data: {
        projectName: PROJECT,
        name: FN_NAME,
        function: "async function TestAutoFunction() { return { ok: true }; }",
        payloadStructure: { bodyType: "json", bodyParams: [] },
        expectedResponseSchema: { rootType: "object", fields: [{ field: "ok", type: "boolean", isActive:
true }] } },
    },
    });
    expect(res.status()).toBe(200);
  });

  test("TC-REG-02 | duplicate function name returns conflict", async () => {
    const ctx = await request.newContext();
    const res = await ctx.post(`${BASE}/forms/createfunction`, {
      data: { projectName: PROJECT, name: FN_NAME, function: "async function TestAutoFunction() {}" },
    });
    const body = await res.json();
    expect(body.ok ?? body.success).toBeFalsy();
  });

  test("TC-REG-03 | getfunction returns correct document", async () => {
    const ctx = await request.newContext();
    const res = await ctx.post(`${BASE}/forms/getfunction`, {
      data: { projectName: PROJECT, fName: FN_NAME },
    });
    expect(res.status()).toBe(200);
    const body = await res.json();
    expect(body.data.name).toBe(FN_NAME);
  });

  test("TC-REG-04 | getAllfunction lists registered functions", async () => {
    const ctx = await request.newContext();
    const res = await ctx.post(`${BASE}/forms/getAllfunction`, {
      data: { projectName: PROJECT },
    });
    const body = await res.json();
    const names = (body.data ?? []).map((f: any) => f.name);
    expect(names).toContain(FN_NAME);
  });
});

```

## Appendix D – playwright.config.ts (reference)

```

// playwright.config.ts
import { defineConfig } from "@playwright/test";

export default defineConfig({
  testDir: "./tests",
  timeout: 30_000,
  retries: 1,
  reporter: [
    ["html", { open: "never" }],
    ["list"],
  ],
  use: {
    baseURL: process.env.API_BASE_URL ?? "http://localhost:4000",
    extraHTTPHeaders: { "Content-Type": "application/json" },
  },
});

```

```
projects: [  
  { name: "api-tests", testMatch: "**/*.spec.ts" },  
],  
});
```

## Appendix E – package.json (reference)

```
{  
  "name": "hackathon-api-tests",  
  "version": "1.0.0",  
  "scripts": {  
    "test": "playwright test",  
    "test:report": "playwright test --reporter=html && playwright show-report"  
  },  
  "devDependencies": {  
    "@playwright/test": "^1.46.0",  
    "typescript": "^5.4.0",  
    "@types/node": "^20.0.0"  
  }  
}
```

## Appendix F – tsconfig.json (reference)

```
{  
  "compilerOptions": {  
    "target": "ES2020",  
    "module": "commonjs",  
    "strict": true,  
    "esModuleInterop": true,  
    "outDir": "dist"  
  },  
  "include": ["tests/**/*.ts", "playwright.config.ts"]  
}
```

## Appendix G – Sanitisation policy

- Real project names (hrx-new, assetflow) replaced with sample-flow.
- Real ObjectIds replaced with placeholder tokens.
- No production routes, internal file paths, or auth tokens in spec files.
- API\_BASE\_URL always read from env; never hard-coded to a production host.

# Problem Statement ID – PS11: Business Workflow Detection & Diagram Generation

**Note:** All project names, ObjectIds, schema names, and function names in this document are fictional sample data. No real credentials, production identifiers, or proprietary code may appear in student submissions.

## 1. Document Control

| Field           | Value                                                                        |
|-----------------|------------------------------------------------------------------------------|
| Mandatory stack | Node.js (backend), MongoDB (persistence), React.js (visual diagram frontend) |
| Classification  | Internal hackathon use — sample data only                                    |

### 1.1 Purpose

Teams build a Business Workflow Engine: a system that reads project context (forms schemas, custom functions, buttons, button conditions) from MongoDB, accepts a natural-language requirement, detects multiple business workflows, generates a workflow definition document, renders an interactive visual diagram in React.js, and executes the chain of actions by consuming the existing dynamic forms API and custom function API.

## 2. Executive Summary

Business processes are sequences of actions triggered by an event. When an order is placed, a vendor must be notified, an invoice must be created, inventory must be decremented, and a confirmation must be sent. Today each of these steps is wired manually in code.

The hackathon challenge is to automate this in three layers: 1. Detection — read project context and requirement, identify distinct workflow chains and their steps. 2. Diagram — render an interactive, drag-and-drop-ready flowchart in React.js showing each step, its action type, conditions, and success/failure paths. 3. Execution — run the workflow by calling real API routes in the correct order, passing outputs of one step as inputs to the next, and recording a run log.

## 3. Problem Statement

Cross-schema business logic is currently fragmented:

- Custom functions perform single actions but are not chained.
- Button operations (operation API) trigger one query or function, but cannot branch or pass context forward.
- There is no way to visualise the sequence, conditions, or failure paths of a multi-step process.
- Developers duplicate similar orchestration code across projects.

Students must close this gap by building a workflow engine that sits on top of the existing API surface and adds orchestration, context passing, conditional branching, and a live visual diagram.

## 4. Hackathon Objectives

### 4.1 Primary objectives (must achieve)

- Accept projectName + business requirement text as input.
- Detect at least one multi-step workflow from project context and requirement.
- Generate and persist a workflow definition document in MongoDB.
- Render an interactive diagram in React.js showing steps, action types, arrows, and conditions.
- Execute the workflow end-to-end by calling real API routes and recording a run log.

### 4.2 Secondary objectives (strongly recommended)

- Detect multiple distinct workflows from one requirement (e.g. order and refund are two separate chains).
- Support conditional steps: execute step only if a field from the trigger or a previous step matches a value.
- Support onFailure branches: abort, skip, or redirect to an alternate step.
- Show live execution status in the diagram (step colour: success=green, failed=red, skipped=amber, pending=grey).
- Allow a user to manually trigger a workflow with a payload from the UI.

### 4.3 Stretch objectives (bonus)

- Drag-and-drop node editor to reorder steps and modify action types in the diagram.
- Webhook trigger: workflow fires on an incoming HTTP POST.
- Scheduled trigger: workflow runs on a cron schedule.
- Step retry on failure with configurable max retries.
- Workflow versioning: save v2 without overwriting v1.

## 5. Scope

### 5.1 In scope

- Node.js API for workflow detection, definition CRUD, trigger, and run-log retrieval.
- MongoDB collections: workflows, workflow\_runs.
- React.js UI: diagram view, trigger panel, run log.
- Step action types: function (POST /forms/function/:name), formCreate, formUpdate, formDelete, operation (POST /forms/operation).
- Context passing: {{trigger.field}} and {{stepId.field}} template syntax in inputMapping.
- At least three sample workflows: OrderPlaced, AssetRequestApproval, InvoiceSettlement.

### 5.2 Out of scope

- Real client data, production endpoints, or proprietary function bodies.
- Email/SMS sending as a step (stubs / console.log are acceptable for notifications).
- Multi-tenant authentication and role-based workflow access.
- Training a foundation model from scratch.

## 6. System Architecture

| Component           | Runtime                               | Responsibility                                                                           |
|---------------------|---------------------------------------|------------------------------------------------------------------------------------------|
| Workflow Studio UI  | React.js                              | Requirement input, diagram display, trigger panel, run log                               |
| Diagram renderer    | React.js + React Flow (or Mermaid.js) | Interactive DAG of workflow steps with live status colours                               |
| Workflow Engine API | Node.js                               | Detect, generate, persist, trigger, and log workflow runs                                |
| Context Loader      | Node.js + MongoDB                     | Load schemas, functions, buttons, button conditions                                      |
| Workflow Detector   | Node.js + LLM/rules                   | Parse requirement into workflow IR (steps, triggers, actions, conditions)                |
| Workflow Executor   | Node.js                               | Walk steps in order, call API routes, pass context, honour conditions                    |
| Run Logger          | Node.js + MongoDB                     | Record each step result and overall run status                                           |
| Existing Forms API  | Node.js (existing)                    | formCreate, formUpdate, formDelete, function/:name, operation — consumed as step actions |

### 6.1 End-to-end pipeline

Requirement + projectName → Load project context → Detect workflows → Build Workflow IR → Persist workflow document to MongoDB → React.js renders interactive diagram → User triggers workflow with payload → Executor walks steps: resolves inputMapping, calls API route → Run log persisted; diagram updates with live step statuses.

## 7. Data Model

### 7.1 Workflow definition document

| Field             | Type   | Description                                                                             |
|-------------------|--------|-----------------------------------------------------------------------------------------|
| workflowName      | string | Unique name within project (e.g. OrderPlaced)                                           |
| description       | string | Human-readable purpose derived from requirement                                         |
| triggerEvent.type | string | Event that starts the workflow: formCreate   formUpdate   formDelete   manual   webhook |

|                     |         |                                                   |
|---------------------|---------|---------------------------------------------------|
| triggerEvent.schema | string  | Schema name that triggers the event (e.g. orders) |
| steps[]             | array   | Ordered list of step objects (see below)          |
| isActive            | boolean | Whether workflow runs on trigger                  |
| isDeleted           | boolean | Soft delete                                       |

## 7.2 Step object

| Field        | Type          | Description                                                            |
|--------------|---------------|------------------------------------------------------------------------|
| stepId       | string        | Unique within workflow (e.g. step-001)                                 |
| name         | string        | Human label (Notify Vendor, Create Invoice)                            |
| order        | number        | Execution sequence (1-based)                                           |
| actionType   | string enum   | function   formCreate   formUpdate   formDelete   operation            |
| functionName | string        | Required when actionType=function; matches custom function name        |
| schema       | string        | Required when actionType=formCreate/Update/Delete                      |
| formId       | string        | Required when actionType=operation                                     |
| buttonId     | string        | Required when actionType=operation                                     |
| inputMapping | object        | Key-value map; values may use {{trigger.x}} or {{stepId.x}} templates  |
| condition    | object   null | { field, operator, value } — step executes only when condition is true |
| onSuccess    | string   null | Next stepId on success, or null to end workflow                        |
| onFailure    | string        | abort   skip   <stepId to jump to>                                     |

## 7.3 Workflow run document

| Field          | Type     | Description                              |
|----------------|----------|------------------------------------------|
| workflowId     | ObjectId | Reference to workflow definition         |
| triggerPayload | object   | Original payload that fired the workflow |



|               |        |                                                  |
|---------------|--------|--------------------------------------------------|
| status        | string | pending   running   completed   failed   partial |
| stepResults[] | array  | { stepId, status, output, durationMs } per step  |
| startedAt     | date   | Execution start timestamp                        |
| completedAt   | date   | Execution end timestamp (null while running)     |

## 7.4 Workflow intermediate representation (IR)

```
{
  "workflowName": "OrderPlaced",
  "triggerEvent": { "type": "formCreate", "schema": "orders" },
  "detectedSteps": [
    { "order": 1, "name": "Notify Vendor",           "actionType": "function",   "candidate":
    "NotifyVendorOnOrder" },
    { "order": 2, "name": "Create Invoice",           "actionType": "formCreate", "candidate":
    "invoices" },
    { "order": 3, "name": "Update Inventory",        "actionType": "operation",  "candidate": null,
    "note": "requires button config" },
    { "order": 4, "name": "Send Confirmation",       "actionType": "function",   "candidate":
    "SendOrderConfirmation" }
  ],
  "confidence": 0.79,
  "warnings": ["step-003 actionType=operation requires manual formId and buttonId"]
}
```

## 8. Workflow Detection

The core intelligence is detecting what workflows exist from a plain-language requirement combined with the project's existing schemas and functions.

### 8.1 Trigger event detection

| Requirement signal                          | Detected trigger                                     |
|---------------------------------------------|------------------------------------------------------|
| when an order is placed / created           | triggerEvent: { type: formCreate, schema: orders }   |
| when a request is approved / status changes | triggerEvent: { type: formUpdate, schema: <schema> } |
| when a record is deleted                    | triggerEvent: { type: formDelete, schema: <schema> } |
| run manually / on demand                    | triggerEvent: { type: manual }                       |
| on webhook / external event                 | triggerEvent: { type: webhook }                      |

## 8.2 Step action detection

| Requirement signal                                | Detected actionType | Resolution                                      |
|---------------------------------------------------|---------------------|-------------------------------------------------|
| notify vendor / send notification / call function | function            | Match to customFunctionModel by name or keyword |
| create invoice / add a record to X                | formCreate          | Map X to FormsSchemaModel.schemaName            |
| update status / change field value                | formUpdate          | Map to schema + field                           |
| approve / reject via button                       | operation           | Match to buttonModel by formId + role           |
| deduct inventory / run a query                    | operation           | Match to button with type=query                 |

## 8.3 Multiple workflow detection

When a requirement describes more than one independent chain (e.g. 'when an order is placed notify the vendor; also when an order is cancelled trigger a refund'), the detector must split them into separate workflow documents. Each workflow has its own trigger and steps array. Use sentence/clause segmentation to detect workflow boundaries. Return an array of workflow IRs; allow user to accept/modify each in the UI before saving.

## 8.4 Context passing: inputMapping templates

| Template                | Resolves to                                    |
|-------------------------|------------------------------------------------|
| {{trigger._id}}         | The _id from the original trigger payload      |
| {{trigger.projectName}} | projectName from the trigger payload           |
| {{trigger.totalAmount}} | Any field from the trigger payload             |
| {{step-001.vendorId}}   | The vendorId field from the output of step-001 |
| {{step-002._id}}        | The _id of the record created by step-002      |

## 9. Visual Diagram Requirements (React.js)

The workflow diagram is a directed acyclic graph (DAG) rendered in React.js. Recommended libraries: React Flow (reactflow) or Mermaid.js. Teams may use any graph library that produces an interactive result.

### 9.1 Diagram node contract

| Node element | Requirement                       |
|--------------|-----------------------------------|
| Step name    | Displayed prominently inside node |

|                |                                                                   |
|----------------|-------------------------------------------------------------------|
| Action type    | Small badge or label (fn(), CREATE, UPDATE, OP)                   |
| Condition flag | Visual indicator when step.condition is not null                  |
| Status colour  | Idle=blue, pending=grey, success=green, failed=red, skipped=amber |
| Step ID        | Shown on hover or in a tooltip for debugging                      |

## 9.2 Diagram edge contract

| Edge                       | Style                                                           |
|----------------------------|-----------------------------------------------------------------|
| onSuccess path             | Green arrow, animated when the step just completed successfully |
| onFailure path (non-abort) | Red dashed arrow to target step                                 |
| abort / skip               | Shown as label on the source node, no outgoing arrow            |

## 9.3 Trigger panel

- A sidebar or modal where the user can fill in the triggerPayload fields derived from triggerEvent.schema.
- A 'Run Workflow' button calls POST /workflow/trigger/:workflowId.
- After triggering, the diagram polls or streams run status and updates node colours.

## 9.4 Run log panel

- List of step results: stepId, status, durationMs, output summary.
- Show full output JSON on expand.
- Overall status and total duration.

# 10. Functional Requirements

## 10.1 Workflow detection and generation API

| ID      | Priority | Requirement                                                                                         |
|---------|----------|-----------------------------------------------------------------------------------------------------|
| FR-WF01 | Must     | POST /workflow/detect — accept projectName + requirement, return array of workflow IRs.             |
| FR-WF02 | Must     | POST /workflow/create — persist one workflow definition document.                                   |
| FR-WF03 | Must     | GET /workflow/list?projectName= — list all workflows for project.                                   |
| FR-WF04 | Must     | GET /workflow/:workflowId — fetch one workflow definition.                                          |
| FR-WF05 | Must     | PATCH /workflow/:workflowId — update validated workflow definition and enable step editing from UI. |

|         |        |                                                                                                                               |
|---------|--------|-------------------------------------------------------------------------------------------------------------------------------|
| FR-WF06 | Must   | DELETE /workflow/:workflowId — soft delete.                                                                                   |
| FR-WF07 | Should | POST /workflow/detect returns multiple IRs when requirement has multiple chains.                                              |
| FR-WF08 | Should | Detection includes confidence score and warnings array in IR.                                                                 |
| FR-WF09 | Should | POST /workflow/:workflowId/version — create a new version from an existing definition without overwriting the active version. |

## 10.2 Workflow execution API

| ID      | Priority | Requirement                                                                                      |
|---------|----------|--------------------------------------------------------------------------------------------------|
| FR-WF10 | Must     | POST /workflow/trigger/:workflowId — execute workflow with triggerPayload in body.               |
| FR-WF11 | Must     | Executor resolves inputMapping templates before each step call.                                  |
| FR-WF12 | Must     | Executor evaluates step.condition before executing; skips step if false.                         |
| FR-WF13 | Must     | Executor aborts remaining steps on onFailure=abort.                                              |
| FR-WF14 | Must     | Executor skips step on onFailure=skip and continues.                                             |
| FR-WF15 | Must     | Persist workflow_run document after each step and on completion.                                 |
| FR-WF16 | Must     | GET /workflow/runs/:workflowId — list run history with stepResults.                              |
| FR-WF17 | Should   | GET /workflow/run/:runId — fetch one run document for diagram status overlay.                    |
| FR-WF18 | Could    | POST /workflow/trigger/:workflowId supports dry-run=true: resolve mappings without calling APIs. |

## 10.3 React.js frontend

| ID      | Priority | Requirement                                                                   |
|---------|----------|-------------------------------------------------------------------------------|
| FR-UI01 | Must     | Requirement input form with projectName and requirement text.                 |
| FR-UI02 | Must     | Display detected workflow IR(s) as draft cards for user to accept or discard. |
| FR-UI03 | Must     | On accept, save workflow and render interactive diagram.                      |
| FR-UI04 | Must     | Diagram nodes show step name, action type, and condition indicator.           |
| FR-UI05 | Must     | Trigger panel: payload form derived from triggerEvent.schema fields.          |
| FR-UI06 | Must     | Run workflow button calls /workflow/trigger/:workflowId.                      |

|         |        |                                                                                                                              |
|---------|--------|------------------------------------------------------------------------------------------------------------------------------|
| FR-UI07 | Must   | Diagram updates step colours from run result after execution.                                                                |
| FR-UI08 | Should | Run log panel with per-step output and duration.                                                                             |
| FR-UI09 | Should | Multiple workflow cards shown side by side when multiple chains detected.                                                    |
| FR-UI10 | Must   | Interactive editor supports drag-and-drop step reordering and editing of step properties; changes are validated before save. |

## 11. API Specification (minimum)

### 11.1 POST /workflow/detect

POST /workflow/detect  
Content-Type: application/json

```
{
  "projectName": "sample-flow",
  "requirement": "When an order is placed, notify the vendor, create an invoice, update inventory, then send a confirmation to the customer.",
  "examplePayload": { "projectName": "sample-flow", "totalAmount": 5000, "stock_type": "physical" }
}
```

Sample response:

```
{
  "ok": true,
  "data": [
    {
      "workflowName": "OrderPlaced",
      "triggerEvent": { "type": "formCreate", "schema": "orders" },
      "detectedSteps": [ /* array of step IR objects */ ],
      "confidence": 0.79,
      "warnings": []
    }
  ]
}
```

### 11.2 POST /workflow/trigger/:workflowId

POST /workflow/trigger/<placeholder-workflow-id>  
Content-Type: application/json

```
{
  "projectName": "sample-flow",
  "_id": "<placeholder-order-id>",
  "totalAmount": 5000,
  "stock_type": "physical"
}
```

Sample response:

```
{
  "ok": true,
  "data": {
    "runId": "<placeholder-run-id>",
    "status": "completed",
    "stepResults": [
```

```

    { "stepId": "step-001", "status": "success", "durationMs": 120 },
    { "stepId": "step-002", "status": "success", "durationMs": 95 },
    { "stepId": "step-003", "status": "skipped", "reason": "condition not met" },
    { "stepId": "step-004", "status": "success", "durationMs": 210 }
  ]
}
}

```

## 12. Sample Hackathon Scenarios

### 12.1 Scenario A — Order placed (primary demo)

| Step              | Action type | Consuming API                                 | Condition              |
|-------------------|-------------|-----------------------------------------------|------------------------|
| Notify Vendor     | function    | POST<br>/forms/function/NotifyVendorOnOrder   | none                   |
| Create Invoice    | formCreate  | POST /forms/formCreate/invoices               | none                   |
| Update Inventory  | operation   | POST /forms/operation                         | stock_type == physical |
| Send Confirmation | function    | POST<br>/forms/function/SendOrderConfirmation | none                   |

### 12.2 Scenario B — Asset request approval (secondary)

| Step                  | Action type | Consuming API                                   | Condition                     |
|-----------------------|-------------|-------------------------------------------------|-------------------------------|
| Validate Request      | function    | POST<br>/forms/function/ValidateAssetRequest    | none                          |
| Notify Approver       | function    | POST<br>/forms/function/NotifyApproverOnRequest | none                          |
| Update Request Status | formUpdate  | POST /forms/formUpdate/asset_requests           | approver_response == approved |
| Create Asset Record   | formCreate  | POST /forms/formCreate/assets                   | approver_response == approved |
| Reject and Notify     | function    | POST /forms/function/NotifyRejection            | approver_response == rejected |

### 12.3 Scenario C — Invoice settlement (stretch)

| Step           | Action type | Consuming API                      | Condition |
|----------------|-------------|------------------------------------|-----------|
| Verify Invoice | function    | POST /forms/function/VerifyInvoice | none      |

|                        |            |                                     |                            |
|------------------------|------------|-------------------------------------|----------------------------|
| Mark Invoice Paid      | formUpdate | POST /forms/formUpdate/invoices     | payment_status == received |
| Release Vendor Payment | function   | POST /forms/function/ReleasePayment | payment_status == received |
| Generate Receipt       | formCreate | POST /forms/formCreate/receipts     | none                       |

## 13. Non-Functional Requirements

| ID     | Area            | Requirement                                                                                            |
|--------|-----------------|--------------------------------------------------------------------------------------------------------|
| NFR-01 | Performance     | Workflow detection responds within 60 seconds including LLM call.                                      |
| NFR-02 | Performance     | Executor completes a 4-step workflow within 5 seconds excluding external latency.                      |
| NFR-03 | Reliability     | If one step fails and onFailure=skip, remaining steps still execute.                                   |
| NFR-04 | Reliability     | Partial run documents are persisted even if workflow aborts mid-way.                                   |
| NFR-05 | Security        | No secrets in workflow documents or run logs. API keys in .env only.                                   |
| NFR-06 | Security        | inputMapping templates must not allow injection of arbitrary keys outside the context object.          |
| NFR-07 | Usability       | Diagram must render within 2 seconds of loading workflow document.                                     |
| NFR-08 | Compatibility   | React.js 18+, Node.js 18+, MongoDB 5+.                                                                 |
| NFR-09 | Maintainability | README with seed data script, sample requirement, and demo steps.                                      |
| NFR-10 | Safety          | Agent editing must be approval-gated; agent proposals must not directly publish or execute a workflow. |
| NFR-11 | Reliability     | Invalid manual or agent edits must leave the currently saved/published workflow unchanged.             |
| NFR-12 | Consistency     | Manual and agent editing must use the same canonical Workflow Definition and validation rules.         |
| NFR-13 | Auditability    | Workflow versions must record edit source and change summary for approved edits.                       |

## 14. Deliverables

- 1. Node.js backend: workflow detect, CRUD, trigger, and run-log APIs.
- 2. React.js frontend: requirement input, workflow diagram, trigger panel, run log.

- 3. MongoDB seed script: at least 2 schemas, 2 custom functions, and sample records for demo.
- 4. At least 2 generated and executed workflows demonstrated live.
- 5. Workflow definition JSON export (sanitized) for each demo workflow.
- 6. README: architecture, setup, seed, and 8-minute judge demo script.
- 7. .env.example with dummy values only.

## 15. Evaluation Rubric (100 points)

| Criterion                   | Points | Judging focus                                                                              |
|-----------------------------|--------|--------------------------------------------------------------------------------------------|
| Workflow detection accuracy | 20     | Correct trigger, steps, action types, and conditions from requirement                      |
| Diagram quality             | 20     | Node labels, action badges, condition flags, edge colours, live status update              |
| Execution correctness       | 25     | All steps called in order, context passed forward, conditions evaluated, run log persisted |
| API consumption             | 15     | Reuses existing formCreate/function/operation routes as step actions                       |
| Multiple workflows          | 10     | Detects and renders more than one independent chain                                        |
| Code quality + docs         | 5      | Readable code, README, seed script, no secrets                                             |
| Innovation / stretch        | 5      | Drag-drop editor, retry, schedule, webhook, versioning                                     |

**Important:** A workflow with steps that are all hard-coded (no context passing, no condition evaluation) cannot score above 40 points. The executor must dynamically resolve inputMapping templates.

## 16. Suggested Timeline (8-hour sprint)

| Hours     | Focus                                                                    |
|-----------|--------------------------------------------------------------------------|
| 0 – 1     | Seed MongoDB, build context loader, implement /workflow/detect skeleton  |
| 1 – 2.5   | Workflow detector: trigger + step IR for Scenario A (OrderPlaced)        |
| 2.5 – 4   | Executor: inputMapping resolver, condition evaluator, API calls, run log |
| 4 – 5.5   | React.js diagram: render nodes + edges from workflow doc; static first   |
| 5.5 – 6.5 | Trigger panel + live status overlay from run result                      |
| 6.5 – 7.5 | Second workflow (Scenario B), multi-workflow UI cards                    |
| 7.5 – 8   | README, demo dry-run, remove secrets                                     |



## Appendix A — Sample Workflow Definition Document (sanitized)

```
{
  "_id": { "$oid": "aaaaaaaaaaaaaaaaaaaaaa" },
  "projectName": "sample-flow",
  "workflowName": "OrderPlaced",
  "description": "Triggered when a new order is created. Notifies vendor, creates invoice, updates
inventory.",
  "triggerEvent": {
    "type": "formCreate",
    "schema": "orders"
  },
},
"steps": [
  {
    "stepId": "step-001",
    "name": "Notify Vendor",
    "order": 1,
    "actionType": "function",
    "functionName": "NotifyVendorOnOrder",
    "inputMapping": { "orderId": "{{trigger._id}}", "projectName": "{{trigger.projectName}}" },
    "condition": null,
    "onSuccess": "step-002",
    "onFailure": "abort"
  },
  {
    "stepId": "step-002",
    "name": "Create Invoice",
    "order": 2,
    "actionType": "formCreate",
    "schema": "invoices",
    "inputMapping": {
      "order_id": "{{trigger._id}}",
      "vendor_id": "{{step-001.vendorId}}",
      "amount": "{{trigger.totalAmount}}"
    },
    "condition": null,
    "onSuccess": "step-003",
    "onFailure": "abort"
  },
  {
    "stepId": "step-003",
    "name": "Update Inventory",
    "order": 3,
    "actionType": "operation",
    "formId": "<placeholder-form-id>",
    "buttonId": "<placeholder-button-id>",
    "inputMapping": { "id": "{{trigger.item_id}}", "projectName": "{{trigger.projectName}}" },
    "condition": { "field": "{{trigger.stock_type}}", "operator": "eq", "value": "physical" },
    "onSuccess": "step-004",
    "onFailure": "skip"
  },
  {
    "stepId": "step-004",
    "name": "Send Confirmation Notification",
    "order": 4,
    "actionType": "function",
    "functionName": "SendOrderConfirmation",
    "inputMapping": { "orderId": "{{trigger._id}}", "invoiceId": "{{step-002._id}}" },
    "condition": null,
    "onSuccess": null,
    "onFailure": "abort"
  }
]
```

```

    }
  ],
  "isActive": true,
  "isDeleted": false,
  "createdAt": { "$date": "2026-08-18T00:00:00.000Z" }
}

```

## Appendix B — Sample Workflow Run Document (sanitized)

```

{
  "_id": { "$oid": "bbbbbbbbbbbbbbbbbbbbbbbbbb" },
  "workflowId": { "$oid": "aaaaaaaaaaaaaaaaaaaaaa" },
  "workflowName": "OrderPlaced",
  "projectName": "sample-flow",
  "triggerPayload": { "_id": "<order-record-id>", "projectName": "sample-flow", "totalAmount": 5000 },
  "status": "completed",
  "stepResults": [
    { "stepId": "step-001", "status": "success", "output": { "vendorId": "<vendor-id>" }, "durationMs": 120 },
    { "stepId": "step-002", "status": "success", "output": { "_id": "<invoice-id>" }, "durationMs": 95 },
    { "stepId": "step-003", "status": "skipped", "reason": "condition not met", "durationMs": 5 },
    { "stepId": "step-004", "status": "success", "output": { "notified": true }, "durationMs": 210 }
  ],
  "startedAt": { "$date": "2026-08-18T07:00:00.000Z" },
  "completedAt": { "$date": "2026-08-18T07:00:00.431Z" }
}

```

## Appendix C — Sample React Diagram Component (sanitized)

```

// WorkflowDiagram.tsx (React + React Flow – sanitized sample)
import React from "react";
import ReactFlow, { Background, Controls, Node, Edge } from "reactflow";
import "reactflow/dist/style.css";

interface WorkflowStep {
  stepId: string;
  name: string;
  order: number;
  actionType: "function" | "formCreate" | "formUpdate" | "operation";
  onSuccess: string | null;
  onFailure: "abort" | "skip" | string | null;
  condition: object | null;
}

interface WorkflowDiagramProps {
  steps: WorkflowStep[];
  runResults?: Record<string, "success" | "failed" | "skipped" | "pending">;
}

const STATUS_COLORS: Record<string, string> = {
  success: "#22c55e",
  failed: "#ef4444",
  skipped: "#f59e0b",
  pending: "#94a3b8",
};

const ACTION_ICONS: Record<string, string> = {

```

```

function: "fn()",
formCreate: "CREATE",
formUpdate: "UPDATE",
operation: "OP",
};

function buildGraph(steps: WorkflowStep[], runResults: Record<string, string> = {}) {
  const nodes: Node[] = steps.map((step, i) => ({
    id: step.stepId,
    position: { x: 250, y: i * 120 },
    data: {
      label: (
        <div style={{ textAlign: "center" }}>
          <div style={{ fontWeight: 700, fontSize: 13 }}><step.name></div>
          <div style={{ fontSize: 11, color: "#6b7280" }}>
            {ACTION_ICONS[step.actionType] ?? step.actionType}
          </div>
          {step.condition && (
            <div style={{ fontSize: 10, color: "#f59e0b" }}>[ conditional ]</div>
          )}
        </div>
      ),
    },
    style: {
      background: runResults[step.stepId]
        ? STATUS_COLORS[runResults[step.stepId]]
        : "#dbeafe",
      border: "1.5px solid #3b82f6",
      borderRadius: 8,
      padding: 8,
      minWidth: 150,
    },
  })),
  const edges: Edge[] = [];
  steps.forEach((step) => {
    if (step.onSuccess) {
      edges.push({
        id: `e-${step.stepId}-success`,
        source: step.stepId,
        target: step.onSuccess,
        label: "success",
        style: { stroke: "#22c55e" },
        animated: runResults[step.stepId] === "success",
      });
    }
    if (step.onFailure && step.onFailure !== "abort" && step.onFailure !== "skip") {
      edges.push({
        id: `e-${step.stepId}-fail`,
        source: step.stepId,
        target: step.onFailure,
        label: "failure",
        style: { stroke: "#ef4444" },
      });
    }
  });
  return { nodes, edges };
}

export const WorkflowDiagram: React.FC<WorkflowDiagramProps> = ({ steps, runResults }) => {
  const { nodes, edges } = buildGraph(steps, runResults ?? {});
  return (
    <div style={{ width: "100%", height: 600 }}>
      <ReactFlow nodes={nodes} edges={edges} fitView>
    </div>
  );
}

```

```

        <Background />
        <Controls />
    </ReactFlow>
</div>
);
};

```

## Appendix D — Sample Workflow Executor Service (sanitized)

```

// workflowExecutor.ts (Node.js service – sanitized)
import axios from "axios";

const API_BASE = process.env.API_BASE_URL ?? "http://localhost:4000";
const PROJECT = process.env.PROJECT_NAME ?? "sample-flow";

type StepResult = { output: Record<string, any>; status: "success" | "failed" | "skipped" };

function resolveMapping(template: Record<string, string>, context: Record<string, any>):
Record<string, any> {
    const result: Record<string, any> = {};
    for (const [key, val] of Object.entries(template)) {
        if (typeof val === "string" && val.startsWith("{") && val.endsWith("}")) {
            const path = val.slice(2, -2).trim();
            const [ns, ...rest] = path.split(".");
            result[key] = rest.reduce((o: any, k) => o?.[k], context[ns]);
        } else {
            result[key] = val;
        }
    }
    return result;
}

function evaluateCondition(condition: any, context: Record<string, any>): boolean {
    if (!condition) return true;
    const [ns, ...rest] = condition.field.replace("{", "").replace("}", "").trim().split(".");
    const actual = rest.reduce((o: any, k) => o?.[k], context[ns]);
    if (condition.operator === "eq") return actual === condition.value;
    if (condition.operator === "neq") return actual !== condition.value;
    if (condition.operator === "gt") return Number(actual) > Number(condition.value);
    return true;
}

export async function executeWorkflow(workflow: any, triggerPayload: any): Promise<StepResult[]> {
    const context: Record<string, any> = { trigger: triggerPayload };
    const results: StepResult[] = [];

    for (const step of workflow.steps) {
        if (!evaluateCondition(step.condition, context)) {
            results.push({ output: {}, status: "skipped" });
            context[step.stepId] = {};
            continue;
        }

        const input = resolveMapping(step.inputMapping ?? {}, context);
        let output: Record<string, any> = {};
        let status: "success" | "failed" = "success";

        try {
            if (step.actionType === "function") {
                const res = await axios.post(`${API_BASE}/forms/function/${step.functionName}`, {
                    projectName: PROJECT, ...input,
                });
            }
        } catch {
            status = "failed";
        }

        results.push({ output, status });
    }

    return results;
}

```

```

    });
    output = res.data?.data ?? {};
  } else if (step.actionType === "formCreate") {
    const res = await axios.post(`${API_BASE}/forms/formCreate/${step.schema}`, {
      projectName: PROJECT, ...input,
    });
    output = res.data?.data ?? {};
  } else if (step.actionType === "formUpdate") {
    const res = await axios.post(`${API_BASE}/forms/formUpdate/${step.schema}`, {
      projectName: PROJECT, ...input,
    });
    output = res.data?.data ?? {};
  } else if (step.actionType === "operation") {
    const res = await axios.post(`${API_BASE}/forms/operation`, {
      projectName: PROJECT, formId: step.formId, buttonId: step.buttonId, ...input,
    });
    output = res.data?.data ?? {};
  }
} catch (err: any) {
  status = "failed";
  if (step.onFailure === "abort") throw new Error(`Workflow aborted at step ${step.stepId}: ${err.message}`);
}

context[step.stepId] = output;
results.push({ output, status });
}

return results;
}

```

## Appendix E — Judge Demo Script (8 minutes)

- 1. Show seeded schemas (orders, invoices, assets) in MongoDB (30s).
- 2. POST /workflow/detect with OrderPlaced requirement; show IR (1.5 min).
- 3. Accept workflow in UI; diagram renders with 4 nodes and arrows (1 min).
- 4. Fill trigger panel with sample order payload and click Run Workflow (30s).
- 5. Watch diagram update: step-001 green, step-002 green, step-003 amber (skipped), step-004 green (1.5 min).
- 6. Open run log; show step output and duration (30s).
- 7. Switch to Scenario B (AssetRequestApproval); show second diagram (1 min).
- 8. Show README and known limitations (remaining time).

## Appendix F — Sanitisation Policy

- Real project names replaced with sample-flow.
- Real ObjectIds replaced with placeholder tokens.
- Real function bodies replaced with /\* generated logic \*/ stubs.
- No production API hosts; API\_BASE\_URL always from environment.
- No real customer names, amounts, or identifiers in seed data.

## 17. Workflow Editing — Agent or Manual

The workflow generated by detection must not be treated as immutable. After the detector produces workflow IRs and the React diagram is rendered, the user must be able to modify the workflow either manually through the visual editor or through a natural-language workflow-editing agent. Both modes must produce the same validated Workflow Definition structure so the existing executor can run the result.

### 17.1 Editing modes

| Mode           | How it works                                                            | Expected result                                |
|----------------|-------------------------------------------------------------------------|------------------------------------------------|
| Manual editing | User edits the workflow directly in the React.js diagram/editor.        | A validated workflow draft.                    |
| Agent editing  | User gives a natural-language instruction against the current workflow. | A proposed patch and updated draft for review. |
| Hybrid editing | Agent proposes a change and the user fine-tunes it manually.            | One final validated workflow draft.            |

### 17.2 Manual workflow editor

The React.js Workflow Studio should provide an Edit mode separate from Run mode. Editing must never execute the workflow.

- Select a node and open a step properties panel.
- Edit step name and order.
- Change `actionType` only to `function`, `formCreate`, `formUpdate`, `formDelete`, or `operation`.
- For function steps, select/edit `functionName` from the known project function catalog.
- For form actions, select the schema from the project schema catalog.
- For operation steps, select `formId` and `buttonId` from known button configuration.
- Edit `inputMapping` using only `{{trigger.field}}` and `{{stepId.field}}` references.
- Create, edit, or remove conditions using `field`, `operator`, and `value`.
- Change `onSuccess` and `onFailure` routing using valid step IDs.
- Add, delete, disable, or duplicate steps.
- Drag and drop nodes to reorder the flow while keeping graph references valid.
- Add or remove valid success/failure connections.
- Edit the trigger type/schema from the supported trigger set.
- Validate before saving; provide `Save Draft`, `Discard Changes`, and `Publish/Save Version` actions.

### 17.3 Agent-based workflow editing

The editing agent operates on the current workflow definition plus relevant project context. It must modify the existing workflow rather than regenerate an unrelated workflow from scratch.

Example natural-language edit commands:

- Add an approval step after `Validate Request`.
- Change `Update Inventory` so it runs only when `stock_type` is `physical`.
- Replace `Send Confirmation` with the `SendOrderConfirmation` function.
- Move `Create Invoice` before `Notify Vendor`.

- Add a rejection branch after Approve Request and call NotifyRejection.
- Make the workflow stop if invoice creation fails.
- Rename step-003 to Update Physical Inventory without changing its action.
- Add a formCreate step for receipts after payment is released.

Agent editing pipeline:

- 1. Load the current workflow definition by workflowId.
- 2. Load relevant project context: schemas, custom functions, buttons, button conditions, and supported action types.
- 3. Interpret the natural-language edit request.
- 4. Generate a structured edit patch rather than directly overwriting the workflow.
- 5. Apply the patch to a copy of the current workflow.
- 6. Validate schema, references, mappings, conditions, and DAG/cycle constraints.
- 7. Return the proposed workflow, warnings, confidence, and a human-readable change summary.
- 8. Display the proposal in the React UI as a before/after preview.
- 9. Require explicit user approval before persistence.
- 10. Persist the approved result as a draft or new version.

## 17.4 Agent edit contract

```
{
  "workflowId": "<workflow-id>",
  "baseVersion": 3,
  "changes": [
    {
      "op": "addStep",
      "afterStepId": "step-002",
      "step": {
        "stepId": "step-003",
        "name": "Approve Request",
        "order": 3,
        "actionType": "operation",
        "formId": "<known-form-id>",
        "buttonId": "<known-approve-button-id>",
        "inputMapping": { "id": "{{trigger._id}}" },
        "condition": null,
        "onSuccess": "step-004",
        "onFailure": "abort"
      }
    }
  ],
  "warnings": [],
  "confidence": 0.94
}
```

## 17.5 Shared validation rules

- Every stepId must be unique within the workflow.
- Every onSuccess/onFailure step reference must point to an existing step.
- The workflow must remain a valid DAG; accidental cycles must be rejected.
- The trigger type must be formCreate, formUpdate, formDelete, manual, or webhook.
- Action-specific fields must be present: functionName for function; schema for form actions; formId and buttonId for operation.

- Known functions, schemas, forms, and buttons should resolve against project context.
- inputMapping may reference only trigger fields or outputs from valid workflow steps.
- Conditions must reference permitted context fields and supported operators.
- Deleting or changing a step must identify and resolve downstream references.
- Agent-generated changes must never introduce arbitrary API endpoints, executable code, secrets, or unsupported action types.
- A validation failure must leave the current saved/published workflow unchanged.

## 17.6 Draft, approval, and versioning model

Use a draft-first model. The active workflow remains executable while an edit is being prepared. Manual and agent edits are applied to a draft. After validation and user approval, the draft can be published or saved as a new version.

| Field         | Type          | Purpose                                     |
|---------------|---------------|---------------------------------------------|
| version       | number        | Sequential workflow version, starting at 1. |
| status        | string        | draft   published   archived.               |
| editSource    | string        | manual   agent   detection.                 |
| baseVersion   | number   null | Version from which an edit was created.     |
| changeSummary | string        | Human-readable summary of the change.       |
| createdBy     | string        | User or system/agent identifier.            |
| updatedBy     | string        | Last editor identifier.                     |
| publishedAt   | date   null   | Time the version became active.             |

## 17.7 API additions for editing

| Method | Endpoint                               | Purpose                                                                                                  |
|--------|----------------------------------------|----------------------------------------------------------------------------------------------------------|
| PATCH  | /workflow/:workflowId                  | Manual editor saves validated workflow changes.                                                          |
| POST   | /workflow/:workflowId/validate         | Validate a draft without publishing it.                                                                  |
| POST   | /workflow/:workflowId/agent-edit       | Accept a natural-language edit request and return a proposed patch/draft; do not persist until approved. |
| POST   | /workflow/:workflowId/agent-edit/apply | Apply an approved agent patch after validation.                                                          |
| POST   | /workflow/:workflowId/version          | Create a new version from the current workflow.                                                          |
| GET    | /workflow/:workflowId/versions         | Return version history and change summaries.                                                             |



|      |                               |                                                                           |
|------|-------------------------------|---------------------------------------------------------------------------|
| POST | /workflow/:workflowId/publish | Publish a validated draft/version.                                        |
| POST | /workflow/:workflowId/draft   | Create or retrieve an editable draft without changing the active version. |

## 17.8 React.js editing experience

| UI state   | Capabilities                                                                                                           |
|------------|------------------------------------------------------------------------------------------------------------------------|
| View / Run | Read-only diagram, trigger panel, run status, and run log.                                                             |
| Edit       | Drag/drop, add/delete/duplicate nodes, edit properties, connections, conditions, mappings, validate and save draft.    |
| Agent Edit | Natural-language command box, proposed change summary, before/after diff, warnings, confidence, Apply/Reject controls. |

Recommended interaction:

- 1. Open a workflow and click Edit Workflow.
- 2. Choose Edit Manually or Edit with Agent.
- 3. For Agent mode, enter the requested change in natural language.
- 4. Show the proposed changes before modifying the saved workflow.
- 5. Highlight added, removed, moved, and modified nodes/edges.
- 6. Show validation errors and warnings next to affected nodes.
- 7. Allow the user to accept, reject, or continue editing the proposal.
- 8. Save as draft/new version and then Publish.
- 9. Only the published/active version should be used for automatic event execution.

## 17.9 Auditability and safety

- Record whether each change came from detection, manual editing, or agent editing.
- Store change summary and base version so edits can be traced.
- Never let an agent directly trigger a workflow while editing.
- Use dry-run validation before publishing.
- Keep secrets and credentials outside workflow documents and agent prompts.
- Reject stale updates or require rebase/merge if the base version is no longer current.

## 17.10 Updated functional requirements

| ID      | Priority | Requirement                                                                                                            |
|---------|----------|------------------------------------------------------------------------------------------------------------------------|
| FR-WF19 | Must     | Workflow edits must be validated before persistence.                                                                   |
| FR-WF20 | Must     | POST /workflow/:workflowId/agent-edit accepts a natural-language edit request and returns a proposed structured patch. |

|         |        |                                                                                                                |
|---------|--------|----------------------------------------------------------------------------------------------------------------|
| FR-WF21 | Must   | Agent-generated edits require explicit user approval before persistence.                                       |
| FR-WF22 | Must   | Manual UI editing supports step properties, ordering, conditions, input mappings, and success/failure routing. |
| FR-WF23 | Must   | Manual and agent editing use the same canonical workflow definition format used by the executor.               |
| FR-WF24 | Must   | Invalid edits must not modify the currently saved/published workflow.                                          |
| FR-WF25 | Should | Edited workflows support draft and published states.                                                           |
| FR-WF26 | Should | Approved edits are traceable to a version and edit source.                                                     |
| FR-WF27 | Should | UI shows a before/after diff for agent-generated edits.                                                        |
| FR-WF28 | Could  | Natural-language undo/revert is supported.                                                                     |

## 17.11 Updated demo flow

- 1. Detect OrderPlaced from the business requirement.
- 2. Accept the detected IR and render the four-step diagram.
- 3. Open Edit Workflow.
- 4. Manually change the inventory condition, validate, and save as a draft.
- 5. Use Agent Edit: “Add an approval step before Send Confirmation.”
- 6. Review the proposed patch and visual before/after diff.
- 7. Approve the agent change and publish the new workflow version.
- 8. Run the published workflow with the trigger payload.
- 9. Show the run log and confirm the edited graph is the graph executed.

## 18. Recommended implementation priority

Implement the editing foundation first and keep agent editing constrained to structured patches. Recommended order: manual editor → shared validation engine → draft/version model → agent patch generation → agent preview/approval → publish → execution. This prevents two different workflow representations.

| Priority | Capability                           | Reason                                                               |
|----------|--------------------------------------|----------------------------------------------------------------------|
| P0       | Shared workflow validator            | Both manual and agent edits depend on the same correctness rules.    |
| P0       | Manual node/edge/property editor     | Provides a deterministic fallback and demonstrates editability.      |
| P0       | Draft + publish/version model        | Prevents unfinished edits from changing the active workflow.         |
| P1       | Agent natural-language edit endpoint | Adds intelligent editing without replacing the deterministic editor. |

|    |                          |                                                                 |
|----|--------------------------|-----------------------------------------------------------------|
| P1 | Agent patch preview/diff | Keeps the user in control and makes changes auditable.          |
| P1 | Agent apply + validation | Converts approved edits into the canonical workflow definition. |
| P2 | Undo/revert/merge        | Useful enhancement after core editing is stable.                |

19. Revision History

| Version | Date        | Change                                                                                                                                                                  |
|---------|-------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1.0     | August 2026 | Original Business Workflow Detection & Diagram Generation requirements.                                                                                                 |
| 1.1     | August 2026 | Added manual workflow editing and agent-based natural-language editing with validation, draft/approval flow, versioning, APIs, UI requirements, safety, and audibility. |

# Problem Statement ID – PS12: Universal AI Chat Interface

**Note:** All project names, ObjectIds, schema names, function names, API URLs, and tokens in this document are fictional. No real credentials, production hosts, or proprietary data may appear in student submissions.

## 1. Document Control

| Field           | Value                                                                 |
|-----------------|-----------------------------------------------------------------------|
| Mandatory stack | Node.js (backend), MongoDB (context + history), React.js (chat UI)    |
| Key distinction | Domain-agnostic: works for any project regardless of schema or domain |

### 1.1 Purpose

Teams build a Universal AI Chat Interface: a single chat window that can serve any project by loading its MongoDB context (schemas, custom functions, workflows, and registered third-party API collections). A user can type in plain language 'show me today's orders', 'place an order for item SKU-001', 'generate a bar chart of revenue by region', 'what is the total unpaid invoice amount' — and the system identifies the intent, calls the correct API or function, and returns the response as text, a table, a bar chart, a line graph, or a pie chart.

### 1.2 Why domain-agnostic matters

The chat must not be hard-coded to any domain. Given a fresh projectName, it reads the project's schemas from MongoDB, discovers available functions, and adapts its capabilities automatically. A chat session for an e-commerce project should handle orders and products. The same system, pointed at an asset-management project, should handle asset requests and approvals without code changes.

## 2. Executive Summary

The chat interface is a thin orchestration layer that sits above every other topic's API. It has three jobs: 1. Understand — classify user intent from natural language using project context as the knowledge source. 2. Act — dispatch the intent to the correct API route, custom function, workflow, or third-party endpoint. 3. Respond — return the result as the most useful format: plain text, a data table, a bar/line/pie chart, or a confirmation card.

The system must work for any domain that has MongoDB schemas and at least one API route. It must also accept a third-party API collection definition and add those endpoints to the chat's action vocabulary automatically.

## 3. Problem Statement

Current limitations of form-driven platforms:

- Users must navigate multiple screens to read data, create records, and view analytics.
- Analytical insights require writing custom aggregation queries not accessible to non-developers.
- Third-party APIs (payment gateways, shipping providers, notification services) require custom integration per project.

- There is no single interface that understands business intent across all schemas, functions, and workflows. Students must build a chat interface that removes these barriers: one text box to rule all project data, actions, and analytics.

## 4. Hackathon Objectives

### 4.1 Primary objectives (must achieve)

- Accept a projectName and load its full context at session start.
- Classify user messages into at least: query\_data, mutate\_data, run\_function, analytics, answer\_question.
- Dispatch to the correct API route and return a structured response.
- Render responses as: plain text, paginated data table, bar chart, line chart, pie chart, and confirmation card.
- Persist conversation history in MongoDB per session.

### 4.2 Secondary objectives (strongly recommended)

- Detect and trigger workflows (POST /workflow/trigger/:id) as a single chat action.
- Accept a third-party API collection JSON, store it in MongoDB, and expose those endpoints as chat actions.
- Multi-turn context: refer back to previous query results ('now filter that by region East').
- Ask clarifying questions when intent is ambiguous before acting.
- Show a 'thinking' indicator with intermediate steps (e.g. 'Identifying schema... calling API...').

### 4.3 Stretch objectives (bonus)

- Voice input via Web Speech API -> text -> same intent pipeline.
- Export chat response data as CSV or PDF.
- Suggested follow-up questions based on the current result.
- Multi-project switcher: one chat interface serving multiple projects in tabs.
- Dashboard mode: pin chat responses as persistent chart/table widgets on a canvas.
- Webhook-triggered chat message: workflow step sends a message to the session.

## 5. Scope

### 5.1 In scope

- Node.js chat API: session management, intent routing, action dispatch, response formatting.
- MongoDB collections: chat\_sessions (conversation history), third\_party\_apis (registered external API collections).
- React.js chat UI: message thread, response renderers (text, table, bar, line, pie, confirmation), file/JSON upload for API collection config.
- Intent types: query\_data, mutate\_data, run\_function, analytics, order\_action, answer\_question, configure\_api, trigger\_workflow.
- Third-party API integration: register a collection JSON, map intent keywords to endpoints, call them from chat.

## 5.2 Out of scope

- Real external API calls to production third-party services during the hackathon demo (stubs are acceptable).
- Authentication and multi-user role management.
- Training or fine-tuning a custom LLM.
- Real-time collaborative chat between multiple users.

## 6. Mandatory Technology Stack

| Layer            | Required                       | Notes                                                                  |
|------------------|--------------------------------|------------------------------------------------------------------------|
| Frontend         | React.js 18+                   | Chat thread, response renderers, chart library (Recharts or Chart.js)  |
| Backend          | Node.js (Express/Fastify)      | Chat session API, intent router, action dispatcher, third-party caller |
| Database         | MongoDB                        | Project context (schemas, functions), chat_sessions, third_party_apis  |
| LLM / NLP        | Any hosted or local model      | Intent classification and entity extraction; API key in .env only      |
| Charts           | Recharts, Chart.js, or Victory | Must support bar, line, and pie with dynamic data                      |
| State management | Redux Toolkit or Zustand       | Chat thread, project context, session state                            |
| Env              | .env / .env.example            | API_BASE_URL, MONGODB_URI, LLM_API_KEY (no real values committed)      |

### 6.1 Recommended folder structure

```
hackathon-chat/  
  backend/  
    contextLoader.ts      <- load schemas, functions, workflows, thirdPartyApis  
    intentRouter.ts       <- classify user message into ResolvedIntent  
    actionDispatcher.ts   <- call correct API route / function / workflow  
    responseFormatter.ts  <- shape raw API data into { responseType, data, chartData }  
    thirdPartyRegistry.ts <- store and call registered external APIs  
    sessionService.ts     <- persist conversation in chat_sessions collection  
    routes/chat.ts        <- POST /chat/message, POST /chat/configure-api, GET /chat/history/:sessionId  
  frontend/  
    components/  
      ChatThread.tsx  
      ChatInput.tsx  
      ChatResponseRenderer.tsx <- text | table | bar | line | pie | confirmation  
      ApiConfigUploader.tsx   <- JSON upload for third-party collection  
      ThinkingIndicator.tsx  
    store/  
      chatSlice.ts  
      contextSlice.ts  
  .env.example  
  README.md
```

## 7. Project Context Model

On session start, the backend loads a rich context object that the intent router uses as its knowledge base. This context is the single source of truth for what the chat can do in a given project.

### 7.1 Context sources

| Source           | Collection / route         | Loaded into context as                                          |
|------------------|----------------------------|-----------------------------------------------------------------|
| Forms schemas    | {project}_forms_schemas    | schemas[] with fields, labels, types, mappedTableRef            |
| Custom functions | {project}_custom_functions | functions[] with name, payloadStructure, expectedResponseSchema |
| Workflows        | workflows (from Topic 5)   | workflows[] with name, triggerEvent, steps                      |
| Buttons          | {project}_buttons          | buttons[] with formId, name, type (query/function)              |
| Third-party APIs | third_party_apis           | thirdPartyApis[] with name, endpoints[], intentKeywords         |
| Query responses  | {project}_response_queries | savedQueries[] with name, aggregation pipeline                  |

### 7.2 Data model: third-party API collection

| Field                      | Type     | Description                                                       |
|----------------------------|----------|-------------------------------------------------------------------|
| name                       | string   | Collection name (e.g. SamplePaymentGateway)                       |
| description                | string   | Human-readable purpose                                            |
| baseUrl                    | string   | Base URL read from env — never hard-coded in document             |
| authType                   | string   | bearer   apikey   basic   none                                    |
| authTokenEnvKey            | string   | Name of env variable holding the token (value never stored in DB) |
| endpoints[].name           | string   | Endpoint identifier (e.g. CreatePayment)                          |
| endpoints[].method         | string   | HTTP method                                                       |
| endpoints[].path           | string   | Path template (e.g. /payments/:paymentId)                         |
| endpoints[].intentKeywords | string[] | Phrases that map this endpoint to a user message                  |

|                             |         |                                                |
|-----------------------------|---------|------------------------------------------------|
| endpoints[].requestSchema   | object  | Expected input fields and types for validation |
| endpoints[].responseMapping | object  | Map API response keys to readable field names  |
| isActive                    | boolean | Whether this collection is active for routing  |
| projectName                 | string  | Scoped to a project                            |

### 7.3 Data model: chat session / conversation history

| Field       | Type   | Description                                                      |
|-------------|--------|------------------------------------------------------------------|
| sessionId   | string | Client-generated or UUID                                         |
| projectName | string | Which project this session is for                                |
| messages[]  | array  | { role: user assistant, content, responseType, data, timestamp } |
| createdAt   | date   | Session start                                                    |
| updatedAt   | date   | Last message timestamp                                           |

## 8. Intent Classification and Routing

### 8.1 Supported intent types

| Intent type  | Example user message                            | Action taken                            | Response format   |
|--------------|-------------------------------------------------|-----------------------------------------|-------------------|
| query_data   | Show me all accounts                            | POST /forms/formGet/:schema             | Table             |
| query_data   | Find orders where status is pending             | POST /forms/formGet/:schema with filter | Table             |
| mutate_data  | Create a new account named Example Corp         | POST /forms/formCreate/:schema          | Confirmation card |
| mutate_data  | Update the phone for account id X to 9999999999 | POST /forms/formUpdate/:schema          | Confirmation card |
| run_function | Get asset detail for request id Y               | POST /forms/function/:functionName      | Text / table      |
| analytics    | Show revenue by region as a bar chart           | POST /forms/queryResponse + aggregate   | Bar chart         |
| analytics    | How many orders were placed                     | Count aggregation on orders schema      | Text + number     |



|                  |                                              |                                                 |                      |
|------------------|----------------------------------------------|-------------------------------------------------|----------------------|
|                  | this week?                                   |                                                 |                      |
| analytics        | Sales trend for last 6 months as line chart  | Time-series aggregation                         | Line chart           |
| order_action     | Place an order for 2 units of SKU-001        | POST /forms/formCreate/orders                   | Confirmation card    |
| trigger_workflow | Run the order placed workflow for order id Z | POST /workflow/trigger/:workflowId              | Workflow run summary |
| answer_question  | What fields does the accounts schema have?   | Read from context (no API call)                 | Text                 |
| configure_api    | Add this payment gateway API collection      | POST /chat/configure-api (store JSON)           | Config saved         |
| third_party      | Check payment status for payment id W        | Call registered endpoint via thirdPartyRegistry | Text / card          |
| unknown          | What is the meaning of life?                 | Return clarification question                   | Text                 |

## 8.2 Intent disambiguation

When the classified intent has confidence below 0.6 or the payload is missing a required field, the system must ask a clarifying question instead of acting. Examples:

- User: 'show me the orders' — missing filter, date range. Ask: 'Would you like all orders, or filter by status or date?'
- User: 'place an order' — missing item and quantity. Ask: 'Which item and how many units?'
- User: 'run the notification function' — multiple matching functions. Ask: 'Did you mean NotifyVendorOnOrder or SendOrderConfirmation?'

## 8.3 Multi-turn context

Each new message is resolved with the full conversation history, enabling: 'Now filter that by region East' — refers to the last table result; 'Sort it by date descending' — applies to the active dataset; 'Create a record for the first result' — uses data from a previous query response. The backend passes the last N messages (configurable, default 10) as conversation\_history to the LLM on every turn. The active dataset (last table/chart data) is also included in context.

## 8.4 Third-party API routing

When a registered third-party API has an endpoint whose intentKeywords match the user message, the intent router must prefer that endpoint over a generic answer\_question response. Match is done by keyword similarity (exact substring or embedding-based if available). Extract required requestSchema fields from the user message or ask if missing. Call the endpoint via the thirdPartyRegistry caller, apply responseMapping, and return the result. Auth token is read from the environment variable named in authTokenEnvKey — never from the DB document.

## 9. Response Format Specification

| responseType  | When to use                                                | React renderer                       |
|---------------|------------------------------------------------------------|--------------------------------------|
| text          | Q&A, clarifications, simple scalar answers                 | Markdown-rendered text bubble        |
| table         | List queries, search results, record fetches               | Paginated sortable HTML table        |
| bar           | Category comparisons (revenue by region, orders by status) | Recharts BarChart                    |
| line          | Time-series trends (sales over time, signups per day)      | Recharts LineChart                   |
| pie           | Part-of-whole distributions (order share by product)       | Recharts PieChart                    |
| confirmation  | Create / update / delete success                           | Green card with record summary       |
| workflow_run  | Workflow triggered from chat                               | Mini run log with step statuses      |
| config_saved  | Third-party API collection registered                      | Confirmation card with endpoint list |
| clarification | System needs more info before acting                       | Grey bubble with follow-up question  |

### 9.1 Chart data contract

```
// Single-series (bar / line / pie)
[ { "label": "North", "value": 12400 },
  { "label": "South", "value": 9800 },
  { "label": "East", "value": 15200 },
  { "label": "West", "value": 7300 } ]

// Multi-series (grouped bar / multi-line)
[ { "label": "Jan", "orders": 120, "returns": 8 },
  { "label": "Feb", "orders": 145, "returns": 11 } ]
```

### 9.2 Table data contract

```
{
  "records": [ { "_id": "<id>", "accountname": "Example Corp", "phone": 9999999999 } ],
  "count": 1,
  "page": 1,
  "limit": 20
}
```

## 10. Functional Requirements

### 10.1 Chat session and message API

| ID      | Priority | Requirement                                                                                                        |
|---------|----------|--------------------------------------------------------------------------------------------------------------------|
| FR-CH01 | Must     | POST /chat/message — accept { sessionId, projectName, message } and return { responseType, data, message }.        |
| FR-CH02 | Must     | GET /chat/history/:sessionId — return full conversation history.                                                   |
| FR-CH03 | Must     | Persist every user and assistant message to chat_sessions collection.                                              |
| FR-CH04 | Must     | Load and pass full project context (schemas, functions, workflows, thirdPartyApis) to intent router on every turn. |
| FR-CH05 | Must     | Pass last 10 messages as conversation_history to LLM.                                                              |
| FR-CH06 | Should   | POST /chat/session — create a new session and return sessionId.                                                    |
| FR-CH07 | Should   | DELETE /chat/session/:sessionId — clear history.                                                                   |
| FR-CH08 | Could    | GET /chat/sessions?projectName= — list sessions for project.                                                       |

### 10.2 Intent routing

| ID      | Priority | Requirement                                                                                     |
|---------|----------|-------------------------------------------------------------------------------------------------|
| FR-IR01 | Must     | Classify user message into one of the supported intent types.                                   |
| FR-IR02 | Must     | Extract payload fields (schema, filter, fieldValues, functionName, chartType) from the message. |
| FR-IR03 | Must     | Return clarification when confidence < 0.6 or required payload field is missing.                |
| FR-IR04 | Must     | Match third-party API endpoint by intentKeywords when relevant.                                 |
| FR-IR05 | Should   | Support multi-turn: resolve pronouns and references using conversation history.                 |
| FR-IR06 | Could    | Return intermediate thinking steps as a separate field for UI display.                          |

### 10.3 Action dispatch

| ID      | Priority | Requirement                                                      |
|---------|----------|------------------------------------------------------------------|
| FR-AD01 | Must     | Dispatch query_data to POST /forms/formGet/:schema.              |
| FR-AD02 | Must     | Dispatch mutate_data (create) to POST /forms/formCreate/:schema. |

|         |        |                                                                                      |
|---------|--------|--------------------------------------------------------------------------------------|
| FR-AD03 | Must   | Dispatch mutate_data (update) to POST /forms/formUpdate/:schema.                     |
| FR-AD04 | Must   | Dispatch run_function to POST /forms/function/:functionName.                         |
| FR-AD05 | Must   | Dispatch analytics to POST /forms/queryResponse or build and run aggregate pipeline. |
| FR-AD06 | Should | Dispatch trigger_workflow to POST /workflow/trigger/:workflowId.                     |
| FR-AD07 | Should | Dispatch third_party to registered endpoint via thirdPartyRegistry.                  |
| FR-AD08 | Could  | Dispatch order_action as formCreate shortcut with domain-specific field defaults.    |

## 10.4 Third-party API management

| ID      | Priority | Requirement                                                                                                |
|---------|----------|------------------------------------------------------------------------------------------------------------|
| FR-TP01 | Must     | POST /chat/configure-api — accept a JSON collection definition, validate, and persist to third_party_apis. |
| FR-TP02 | Must     | Load all active third-party API collections at context load time.                                          |
| FR-TP03 | Must     | Auth token fetched from process.env[authTokenEnvKey] — never stored in DB value.                           |
| FR-TP04 | Must     | Map response using responseMapping before returning to frontend.                                           |
| FR-TP05 | Should   | GET /chat/third-party-apis?projectName= — list registered collections.                                     |
| FR-TP06 | Should   | PATCH /chat/third-party-apis/:name — update endpoint definitions.                                          |
| FR-TP07 | Could    | DELETE /chat/third-party-apis/:name — soft-delete a collection.                                            |

## 10.5 React.js frontend

| ID      | Priority | Requirement                                                                               |
|---------|----------|-------------------------------------------------------------------------------------------|
| FR-UI01 | Must     | Chat thread: alternating user and assistant bubbles, auto-scroll to latest.               |
| FR-UI02 | Must     | ChatResponseRenderer switches on responseType: text, table, bar, line, pie, confirmation. |
| FR-UI03 | Must     | Project selector at session start: enter or select projectName.                           |
| FR-UI04 | Must     | Text input with send button; Enter key submits.                                           |
| FR-UI05 | Must     | Loading / thinking indicator while request is in flight.                                  |

|         |        |                                                                                |
|---------|--------|--------------------------------------------------------------------------------|
| FR-UI06 | Must   | Table renderer: paginated, columns derived from record keys.                   |
| FR-UI07 | Should | JSON file upload or paste area for third-party API collection config.          |
| FR-UI08 | Should | Chart tooltips, legend, and responsive container for all chart types.          |
| FR-UI09 | Should | Copy-to-clipboard on code blocks and table data.                               |
| FR-UI10 | Could  | Suggested follow-up questions rendered as clickable chips below each response. |
| FR-UI11 | Could  | Pin button on chart/table responses to add them to a persistent dashboard.     |

## 11. API Specification (minimum)

### 11.1 POST /chat/message

POST /chat/message  
Content-Type: application/json

```
{
  "sessionId": "sess-000000001",
  "projectName": "sample-flow",
  "message": "Show me all orders placed today as a bar chart by status"
}
```

Sample response:

```
{
  "ok": true,
  "data": {
    "responseType": "bar",
    "message": "Here is the order status breakdown for today:",
    "chartData": [
      { "label": "Pending", "value": 12 },
      { "label": "Shipped", "value": 34 },
      { "label": "Delivered", "value": 21 }
    ],
    "intent": { "type": "analytics", "schema": "orders", "chartType": "bar", "confidence": 0.91 }
  }
}
```

### 11.2 POST /chat/configure-api

Accepts a third-party API collection JSON (see Appendix D for schema).

POST /chat/configure-api  
Content-Type: application/json

```
{
  "projectName": "sample-flow",
  "collection": { /* third-party API collection document */ }
}
```

After registration, user can say: 'check payment status for payment id X' and the chat routes to the registered endpoint.

### 11.3 Error model

```
{
  "ok": false,
  "data": {
    "responseType": "clarification",
    "message": "I need more information. Which schema are you asking about?"
  }
}
```

## 12. Sample Chat Scenarios (domain-agnostic)

| User message                                     | Detected intent              | API called                                       | Response format      |
|--------------------------------------------------|------------------------------|--------------------------------------------------|----------------------|
| Show me all accounts                             | query_data / accounts        | POST /forms/formGet/accounts                     | Table                |
| How many orders are there by status? Show as pie | analytics / orders           | Aggregate on orders schema                       | Pie chart            |
| Place an order for 3 units of item SKU-001       | order_action / orders        | POST /forms/formCreate/orders                    | Confirmation         |
| Get asset detail for request id <placeholder>    | run_function                 | POST /forms/function/GetAssetDetail              | Text / table         |
| Run the order placed workflow for order <id>     | trigger_workflow             | POST /workflow/trigger/:workflowId               | Workflow run summary |
| What fields does the invoices schema have?       | answer_question              | Read from context (no API call)                  | Text                 |
| Check payment status for payment <id>            | third_party / PaymentGateway | Registered SamplePaymentGateway/GetPaymentStatus | Text card            |
| Sales trend for last 6 months as line chart      | analytics / orders           | Time-series aggregate                            | Line chart           |
| Update account <id> phone to 8888888888          | mutate_data / accounts       | POST /forms/formUpdate/accounts                  | Confirmation         |
| Add a new payment gateway API collection         | configure_api                | POST /chat/configure-api                         | Config saved         |

## 13. Non-Functional Requirements

| ID     | Area        | Requirement                                                     |
|--------|-------------|-----------------------------------------------------------------|
| NFR-01 | Performance | Chat response (non-LLM path) under 2 seconds. LLM-path under 10 |

|        |                 |                                                                                     |
|--------|-----------------|-------------------------------------------------------------------------------------|
|        |                 | seconds with streaming or progress indicator.                                       |
| NFR-02 | Reliability     | If LLM fails, return a graceful clarification response, not a 500 crash.            |
| NFR-03 | Security        | Third-party auth tokens never stored in MongoDB; always read from env.              |
| NFR-04 | Security        | Chat API must sanitize all user input before passing to DB queries.                 |
| NFR-05 | Privacy         | Conversation history contains no real credentials or sensitive identifiers in demo. |
| NFR-06 | Usability       | Empty-state message when no messages yet. Clear error messages.                     |
| NFR-07 | Compatibility   | Chrome / Edge latest. Node.js 18+. MongoDB 5+.                                      |
| NFR-08 | Scalability     | Session history capped at configurable message limit (default 100 per session).     |
| NFR-09 | Maintainability | README with seed, .env.example, and demo conversation script.                       |

## 14. Deliverables

- 1. Node.js backend: chat API (message, history, configure-api, session routes).
- 2. React.js frontend: chat thread, all response renderers, project selector, API config uploader.
- 3. MongoDB seed: at least 2 schemas, 2 custom functions, 1 registered third-party API collection.
- 4. Demo of at least 5 distinct intent types in a live chat session.
- 5. README: setup, seed, .env.example, and sample conversation demo script.
- 6. At least one chart type (bar, line, or pie) demonstrably driven by a user message.

## 15. Evaluation Rubric (100 points)

| Criterion                      | Points | Judging focus                                                           |
|--------------------------------|--------|-------------------------------------------------------------------------|
| Intent classification accuracy | 20     | Correct intent type, schema, payload extracted from varied messages     |
| Response rendering quality     | 20     | Table, chart (min 2 types), confirmation, and text all working          |
| API consumption correctness    | 20     | Calls correct formGet/formCreate/function/queryResponse/workflow routes |
| Domain-agnosticism             | 15     | Works with a fresh schema set without code changes                      |
| Third-party API integration    | 10     | Register collection, route to it from chat, return mapped response      |
| Multi-turn context             | 10     | Follow-up questions refer to previous results correctly                 |
| Code quality + docs            | 5      | Readable, no secrets, README, demo script                               |

**Important:** A hard-coded chatbot that only answers predefined questions cannot score above 30 points. The intent router must dynamically use project context loaded from MongoDB.

## 16. Suggested Timeline (8-hour sprint)

| Hours     | Focus                                                                      |
|-----------|----------------------------------------------------------------------------|
| 0 – 1     | Seed MongoDB, implement context loader, POST /chat/message skeleton        |
| 1 – 2.5   | Intent router: query_data + mutate_data intents working end-to-end         |
| 2.5 – 4   | React.js chat thread + ChatResponseRenderer: text + table + one chart type |
| 4 – 5.5   | analytics intent: aggregate pipeline, bar + line + pie charts              |
| 5.5 – 6.5 | run_function + third-party API: register collection and call from chat     |
| 6.5 – 7.5 | Multi-turn context, clarification questions, session persistence           |
| 7.5 – 8   | README, demo script, secrets check, dry-run demo                           |

## Appendix A — Intent Router (sanitized Node.js sample)

```
// intentRouter.ts (Node.js – sanitized)
import { loadProjectContext } from "../contextLoader";
import { callLLM } from "../llmClient";

export type IntentType =
  | "query_data"
  | "mutate_data"
  | "run_function"
  | "analytics"
  | "order_action"
  | "answer_question"
  | "configure_api"
  | "unknown";

export interface ResolvedIntent {
  type: IntentType;
  schema?: string;
  functionName?: string;
  payload?: Record<string, any>;
  chartType?: "bar" | "line" | "pie" | "table";
  aggregation?: object;
  thirdPartyRef?: string;
  clarification?: string;
  confidence: number;
}

export async function routeIntent(
  userMessage: string,
```



```

    projectName: string,
    conversationHistory: { role: string; content: string }[]
  ): Promise<ResolvedIntent> {
    const context = await loadProjectContext(projectName);

    const systemPrompt = `
You are an AI assistant for the project "${projectName}".
Available schemas: ${context.schemas.map(s => s.schemaName).join(", ")}.
Available functions: ${context.functions.map(f => f.name).join(", ")}.
Third-party APIs registered: ${context.thirdPartyApis.map(a => a.name).join(", ") || "none"}.

Classify the user message into one of:
query_data, mutate_data, run_function, analytics, order_action, answer_question, configure_api, unknown.

Return JSON:
{
  "type": "<intent>",
  "schema": "<schemaName or null>",
  "functionName": "<name or null>",
  "payload": { /* extracted key-value pairs */ },
  "chartType": "<bar|line|pie|table or null>",
  "aggregation": null,
  "thirdPartyRef": "<api name or null>",
  "clarification": "<question if ambiguous, else null>",
  "confidence": 0.0
};

    const response = await callLLM({
      system: systemPrompt,
      messages: [...conversationHistory, { role: "user", content: userMessage }],
    });

    try {
      return JSON.parse(response) as ResolvedIntent;
    } catch {
      return { type: "unknown", confidence: 0, clarification: "Could not parse intent." };
    }
  }
}

```

## Appendix B — Action Dispatcher (sanitized Node.js sample)

```

// actionDispatcher.ts (Node.js – sanitized)
import axios from "axios";

const API_BASE = process.env.API_BASE_URL ?? "http://localhost:4000";
const PROJECT = process.env.PROJECT_NAME ?? "sample-flow";

export async function dispatchIntent(intent: any, projectName: string): Promise<any> {
  if (intent.type === "query_data" && intent.schema) {
    const res = await axios.post(`${API_BASE}/forms/formGet/${intent.schema}`, {
      projectName,
      searchBy: intent.payload?.searchBy ?? "",
      paginated: true,
      limit: intent.payload?.limit ?? 20,
      offset: intent.payload?.offset ?? 0,
    });
    return { responseType: "table", data: res.data?.data };
  }

  if (intent.type === "mutate_data" && intent.schema) {
    const endpoint = intent.payload?._id ? "formUpdate" : "formCreate";

```

```

const res = await axios.post(`${API_BASE}/forms/${endpoint}/${intent.schema}`, {
  projectName,
  ...intent.payload,
});
return { responseType: "confirmation", data: res.data?.data };
}

if (intent.type === "run_function" && intent.functionName) {
  const res = await axios.post(`${API_BASE}/forms/function/${intent.functionName}`, {
    projectName,
    ...intent.payload,
  });
  return { responseType: "text", data: res.data?.data };
}

if (intent.type === "analytics") {
  const res = await axios.post(`${API_BASE}/forms/queryResponse`, {
    projectName,
    name: intent.payload?.queryName,
  });
  return {
    responseType: intent.chartType ?? "table",
    data: res.data?.data,
    chartType: intent.chartType,
  };
}

if (intent.type === "order_action") {
  const res = await axios.post(`${API_BASE}/forms/formCreate/${intent.schema}`, {
    projectName,
    ...intent.payload,
  });
  return { responseType: "confirmation", data: res.data?.data };
}

if (intent.type === "configure_api") {
  return { responseType: "config_saved", data: { name: intent.thirdPartyRef } };
}

return { responseType: "text", data: "I could not determine what you want to do. Could you rephrase?" };
}

```

## Appendix C — ChatResponseRenderer (sanitized React sample)

```

// ChatResponseRenderer.tsx (React – sanitized)
import React from "react";
import { Bar, Line, Pie } from "recharts";
import { ResponsiveContainer, BarChart, LineChart, PieChart, XAxis, YAxis, Tooltip, Legend, Cell } from
"recharts";

interface ChatResponse {
  responseType: "text" | "table" | "bar" | "line" | "pie" | "confirmation" | "clarification";
  data: any;
  chartType?: string;
  message?: string;
}

const COLORS = ["#3b82f6", "#22c55e", "#f59e0b", "#ef4444", "#8b5cf6", "#06b6d4"];

export const ChatResponseRenderer: React.FC< { response: ChatResponse }> = ({ response }) => {

```

```

const { responseType, data, message } = response;

if (responseType === "text" || responseType === "clarification") {
  return <div className="chat-bubble assistant">{message ?? JSON.stringify(data)}</div>;
}

if (responseType === "confirmation") {
  return (
    <div className="chat-bubble assistant success">
      <strong>Done!</strong> Record created / updated successfully.
      <pre style={{ fontSize: 11, marginTop: 8 }}>{JSON.stringify(data, null, 2)}</pre>
    </div>
  );
}

if (responseType === "table") {
  const records: Record<string, any>[] = Array.isArray(data?.records) ? data.records :
(Array.isArray(data) ? data : []);
  const columns = records.length > 0 ? Object.keys(records[0]) : [];
  return (
    <div style={{ overflowX: "auto", marginTop: 8 }}>
      <table className="chat-table">
        <thead>
          <tr>{columns.map(c => <th key={c}>{c}</th>)}</tr>
        </thead>
        <tbody>
          {records.map((row, i) => (
            <tr key={i}>
              {columns.map(c => <td key={c}>{String(row[c] ?? "")}</td>)}
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  );
}

const chartData: any[] = Array.isArray(data) ? data : [];

if (responseType === "bar" || response.chartType === "bar") {
  return (
    <ResponsiveContainer width="100%" height={300}>
      <BarChart data={chartData}>
        <XAxis dataKey="label" /><YAxis /><Tooltip /><Legend />
        <Bar dataKey="value" fill={COLORS[0]} />
      </BarChart>
    </ResponsiveContainer>
  );
}

if (responseType === "line" || response.chartType === "line") {
  return (
    <ResponsiveContainer width="100%" height={300}>
      <LineChart data={chartData}>
        <XAxis dataKey="label" /><YAxis /><Tooltip /><Legend />
        <Line type="monotone" dataKey="value" stroke={COLORS[0]} />
      </LineChart>
    </ResponsiveContainer>
  );
}

if (responseType === "pie" || response.chartType === "pie") {
  return (
    <ResponsiveContainer width="100%" height={300}>
      <PieChart>

```

```

        <Pie data={chartData} dataKey="value" nameKey="label" cx="50%" cy="50%" outerRadius={100}
label>
        {chartData.map((_,i) => <Cell key={i} fill={COLORS[i % COLORS.length]} />)}
        </Pie>
        <Tooltip /><Legend />
        </PieChart>
    </ResponsiveContainer>
    );
}

return <pre style={{ fontSize: 12 }}>{JSON.stringify(data, null, 2)}</pre>;
};

```

## Appendix D — Third-Party API Collection Document (sanitized)

```

{
  "name": "SamplePaymentGateway",
  "description": "Fictional payment gateway API",
  "baseUrl": "https://api.samplepayment.local/v1",
  "authType": "bearer",
  "authTokenEnvKey": "SAMPLE_PAYMENT_TOKEN",
  "endpoints": [
    {
      "name": "CreatePayment",
      "method": "POST",
      "path": "/payments",
      "intentKeywords": ["pay", "charge", "create payment", "initiate payment"],
      "requestSchema": {
        "amount": "number",
        "currency": "string",
        "orderId": "string"
      },
      "responseMapping": {
        "paymentId": "data.id",
        "status": "data.status"
      }
    },
    {
      "name": "GetPaymentStatus",
      "method": "GET",
      "path": "/payments/:paymentId",
      "intentKeywords": ["payment status", "check payment", "payment details"],
      "requestSchema": { "paymentId": "string" },
      "responseMapping": { "status": "data.status", "amount": "data.amount" }
    }
  ],
  "isActive": true,
  "projectName": "sample-flow"
}

```

## Appendix E — Sample Chat Session Document (sanitized)

```

{
  "_id": { "$oid": "cccccccccccccccccccccccccc" },
  "sessionId": "sess-0000000001",
  "projectName": "sample-flow",
  "messages": [
    {

```

```

    "role": "user",
    "content": "Show me all orders placed today"
  },
  {
    "role": "assistant",
    "content": "Here are today's orders:",
    "responseType": "table",
    "data": []
  },
  {
    "role": "user",
    "content": "How many orders are from each region? Show as a bar chart"
  },
  {
    "role": "assistant",
    "content": "Here is the regional breakdown:",
    "responseType": "bar",
    "data": []
  },
  {
    "role": "user",
    "content": "Place an order for 2 units of item SKU-001"
  },
  {
    "role": "assistant",
    "content": "Order placed successfully.",
    "responseType": "confirmation"
  }
],
"createdAt": { "$date": "2026-08-18T00:00:00.000Z" },
"updatedAt": { "$date": "2026-08-18T00:05:00.000Z" }
}
}

```

## Appendix F — Judge Demo Script (8 minutes)

- 1. Open chat, select project sample-flow (30s).
- 2. Type: 'Show me all accounts' — table renders (1 min).
- 3. Type: 'How many accounts per industry? Show as pie chart' — pie chart renders (1 min).
- 4. Type: 'Create an account named Demo Corp' — confirmation card (1 min).
- 5. Type: 'Get asset detail for request id <seed id>' — function called, text/table response (1 min).
- 6. Upload third-party API collection JSON; type: 'Check payment status for payment id P001' — registered endpoint called (1.5 min).
- 7. Show conversation history persisted in MongoDB (30s).
- 8. Known limitations + questions (remaining time).

## Appendix G — Sanitisation Policy

- Real project names replaced with sample-flow.
- Real ObjectIds replaced with <placeholder-id> tokens.
- Third-party API base URLs use .local / placeholder domains only.
- Auth tokens referenced by env key name only; no values stored or committed.
- No real customer names, phone numbers, or financial amounts in seed data.